

Master Thesis

August 30, 2026

Forgetting by Design

Trade-offs in Machine Unlearning for LLM Code
Generation

Dogukan Baysal

of Girne, Turkish Republic of Northern Cyprus (24-745-069)

supervised by

Prof. Dr. Harald C. Gall

Melih Catal



University of
Zurich ^{UZH}



Master Thesis

Forgetting by Design

Trade-offs in Machine Unlearning for LLM Code
Generation

Dogukan Baysal



University of
Zurich ^{UZH}



Master Thesis

Author: Dogukan Baysal, dogukan.baysal@uzh.ch

Project period: 27.03.2026 - 27.09.2026

Software Evolution & Architecture Lab

Department of Informatics, University of Zurich

Acknowledgements

First of all, I would like to thank my family, Handan, Hakan and Tuana Baysal, who made this journey possible. Without their assistance and encouragement, I would not be where I am today. I would also like to express my sincere gratitude to Melih Catal, who guided me with his knowledge and support throughout both my independent study and thesis. I am thankful to Alex Wolf for patiently bearing with my endless questions, and to Prof. Dr. Harald C. Gall for making this thesis possible. Finally, I would like to thank Seyma Zeynep Incecik for her constant support, endless understanding, and for making this entire process more manageable by being there for me.

Abstract

Large Language Models (LLMs) trained on source code may memorize sensitive or copyrighted content contained in their training data. Therefore, they may generate code that reproduces these patterns, potentially exposing sensitive information such as passwords, API keys, and email addresses, as well as copyrighted code that is not authorized for redistribution. These risks affect not only organizations from legal and security perspectives, but also individual users who may have exposed sensitive information or personally identifiable information (PII) online. Machine Unlearning (MU) aims to solve this problem by removing the influence of such data from a trained model without requiring retraining from scratch. This significantly reduces resource requirements and makes it feasible to handle frequent unlearning requests. Although multiple MU techniques have been proposed, limited research has examined how to apply them in practice for code generation. This prevents LLM providers from making informed decisions about which unlearning objectives and strategies best balance three key dimensions of MU: the quality of the forgetting effect, the retention of coding capabilities, and energy consumption. In this thesis, we categorize unlearning techniques according to their objectives and evaluate them in terms of this three-dimensional trade-off.

Our results show that secrets can be removed while largely preserving model utility when the technique and training configuration are selected carefully. In contrast, the same favorable trade-offs do not transfer to complete code units: approaches that achieve strong forgetting severely degrade utility, whereas approaches that preserve utility yield only limited forgetting. The way the unlearning process is configured also matters: processing the data to be removed before the data to be preserved generally improves utility retention, whereas simply enlarging the preserved data brings no consistent benefit. These findings provide practical guidance for applying unlearning techniques to achieve more favorable outcomes. They also indicate that unlearning behavior depends strongly on the structure of the targeted content and that current MU techniques are not yet suitable for reliably removing complete code units.

Zusammenfassung

LLMs, die auf Quellcode trainiert wurden, können sensible oder urheberrechtlich geschützte Inhalte aus ihren Trainingsdaten memorieren. Daher besteht die Gefahr, dass sie Code generieren, der diese Muster reproduziert und dabei möglicherweise sensible Informationen wie Passwörter, API-Schlüssel und E-Mail-Adressen sowie urheberrechtlich geschützten Code, der nicht zur Weiterverbreitung autorisiert ist, preisgibt. Diese Risiken betreffen nicht nur Organisationen aus rechtlicher und sicherheitstechnischer Sicht, sondern auch einzelne Nutzer:innen, die möglicherweise sensible Informationen oder personenbezogene Daten (PII) online preisgegeben haben. MU zielt darauf ab, dieses Problem zu lösen, indem der Einfluss solcher Daten aus einem trainierten Modell entfernt wird, ohne ein vollständiges Neutrainig zu erfordern. Dadurch werden die benötigten Ressourcen erheblich reduziert und die Bearbeitung häufiger Unlearning-Anfragen wird praktikabler. Obwohl verschiedene MU-Techniken vorgeschlagen wurden, gibt es bislang nur wenig Forschung dazu, wie sie in der Praxis für die Codegenerierung angewendet werden können. Dies erschwert es LLM-Anbietern, fundierte Entscheidungen darüber zu treffen, welche Unlearning-Ziele und -Strategien die drei zentralen Dimensionen von MU am besten ausbalancieren: die Qualität des Vergessenseffekts, den Erhalt der Programmierfähigkeiten und den Energieverbrauch. In dieser Arbeit kategorisieren wir Unlearning-Techniken nach ihren Zielen und evaluieren sie im Hinblick auf diesen dreidimensionalen Zielkonflikt.

Unsere Ergebnisse zeigen, dass sich Geheimnisse entfernen lassen, wobei die Nützlichkeit des Modells weitgehend erhalten bleibt, sofern Technik und Trainingskonfiguration sorgfältig gewählt werden. Im Gegensatz dazu lassen sich diese günstigen Zielkonflikte nicht auf vollständige Codeeinheiten übertragen: Ansätze, die ein starkes Vergessen bewirken, beeinträchtigen die Nützlichkeit erheblich, während Ansätze, die die Nützlichkeit erhalten, nur ein begrenztes Vergessen ermöglichen. Auch die Konfiguration des Unlearning-Prozesses spielt eine Rolle: Werden die zu entfernenden Daten vor den zu erhaltenden Daten verarbeitet, verbessert dies im Allgemeinen den Erhalt der Nützlichkeit, während eine bloße Vergrößerung der Menge der zu erhaltenden Daten keinen konsistenten Vorteil bietet. Diese Erkenntnisse liefern praktische Hinweise für den Einsatz von Unlearning-Techniken, um günstigere Ergebnisse zu erzielen. Sie zeigen außerdem, dass das Unlearning-Verhalten stark von der Struktur der Zielinhalte abhängt und dass aktuelle MU-Techniken noch nicht geeignet sind, vollständige Codeeinheiten zuverlässig zu entfernen.

Contents

1	Introduction	1
2	Related Work and Background	3
2.1	Machine Unlearning	3
2.2	Machine Unlearning Objectives	3
2.2.1	Forget Objectives	4
2.2.2	Retain Objectives	5
2.3	Dataset Curation for Machine Unlearning	6
2.4	Machine Unlearning for Code	6
2.5	Evaluation of Machine Unlearning for Code	7
2.6	Resource Efficiency of Machine Unlearning Techniques	7
3	Methodology	9
3.1	Research Questions	9
3.2	Study Design	10
3.2.1	Benchmark Construction	10
3.2.2	Model Selection	15
3.2.3	Machine Unlearning Objective Selection	15
3.2.4	Evaluation	15
4	Experimental Design	19
4.1	Fine-Tuning	19
4.2	Unlearning	19
4.2.1	Experimental Configurations	20
5	Results and Discussion	23
5.1	RQ1: Machine Unlearning Technique Trade-offs for Secrets	23
5.2	RQ2: Transferability of Machine Unlearning to Code Units	29
5.3	RQ3: Effects of Retain Set Size and Forget–Retain Objective Ordering	31
5.4	Ablation Studies	35
6	Limitations and Future Work	37
7	Conclusion	39

List of Figures

3.1	Overview of the experimental workflow. A pre-trained model is first fine-tuned on the synthetic dataset with injected secrets, then unlearned using the forget and retain sets. The resulting model is evaluated along three dimensions: forget quality, utility retention, and resource consumption.	10
3.2	Overview of the benchmark construction workflow. GPT-5.5 generates candidate functions and classes, secrets are then planted into the forget set. All candidates pass through a similarity evaluation, elements that violate the configured similarity thresholds are regenerated.	11
3.3	Pairwise lexical and structural similarity over all 179,700 pairs of the final dataset (log-scaled counts). Left: direction-averaged CodeBLEU; Right: token Jaccard similarity. Both lie entirely below the 0.6 threshold (dashed red), with maxima of 0.55 and 0.53 respectively	15
3.4	Pairwise cosine similarity over all 179,700 pairs for two code embedding models. Raw cosine (grey) is compressed into a narrow band near 0.6, illustrating embedding anisotropy. After mean-centering (blue), the distribution re-centers near 0. . .	16
4.1	Mean generated–target similarity across the base and five training epochs for both models. Secret memorization is measured with chrF and code unit memorization with BLEU, averaged over 150 examples.	20
4.2	Performance of the fine-tuned models at checkpoint four before unlearning. The left panel shows the percentage of items with at least one exact match among ten generations, while the right reports ForgetEval and UtilityEval pass@10 scores. . .	21
4.3	Trade-off between forgetting quality and model utility across learning rates for Gradient Ascent (GA), Negative Preference Optimization (NPO), and Probabilistic Redistribution for Output Distribution (PROD). Each line corresponds to a learning rate, with points representing different epochs. Higher values on both axes are preferred, making the upper-right region the most favorable area.	22
5.1	Epoch-wise performance of GA variants under secret unlearning. Columns correspond to the different models used in the experiments. Rows report Forget-Quality@10 and UtilityEval pass@10. Higher values indicate better performance for both metrics.	24
5.2	Epoch-wise utility–forgetting trajectories for PROD variants under secret unlearning. The axes report UtilityEval pass@10 and Forget-Quality@10, with higher values preferred. Point labels denote epochs and arrows indicate training direction. .	25
5.3	Utility–forgetting trade-off for the selected secret setting checkpoints. For each model–technique configuration, we consider epochs satisfying the minimum thresholds of 0.80 for both UtilityEval pass@10 and Forget-Quality@10 and select the epoch maximizing their equally weighted mean.	27
5.4	Measured GPU energy for secret unlearning runs. Bars show the mean across Llama and Qwen; error bars span the two model-specific values.	28
5.5	ForgetEval pass@10 failure rates by secret location for the selected secret setting checkpoints. Bars report the mean failure rate across Llama and Qwen, while error bars span the two model-specific rates. For each model–technique pair, the checkpoint maximizing the equally weighted mean of UtilityEval pass@10 and Forget-Quality@10 was selected.	29

5.6	Utility-forgetting trade-offs for all secret and code unit checkpoints across both models. For clarity, the visualization of the retain objective is omitted. Higher values on both axes are preferred.	31
5.7	Effects of Forget First relative to random ordering in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers their means, and horizontal lines the observed ranges.	32
5.8	Effect of Forget First ordering on the GA-family utility-forgetting trade-off. Numbers identify epochs, and lines connect epochs 1–3 for the same model and ordering. The shaded region denotes UtilityEval of at least 0.80.	32
5.9	Effects of Retain First relative to random ordering in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers their means, and horizontal lines the observed ranges.	33
5.10	Effects of doubling retain set size in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers show their means, and horizontal lines show the observed ranges.	34
5.11	GPU energy consumption and training time of standard secret and double retain set unlearning setups. Bars report the mean over Llama and Qwen, while error bars span the two model measurements.	34
1	Learning-rate sensitivity of full-parameter GA on Qwen2.5-Coder-3B under the secret unlearning setting. Connected markers represent epochs 1–3.	48
2	Utility-forgetting trade-offs for all checkpoints across both models, using 1–normalized character-level Levenshtein similarity as a common forget quality measure. For clarity, the visualization of the retain objective is omitted. Higher values on both axes are preferred.	49

List of Tables

3.1	Overview of the dataset composition. Each component is divided equally between functions and classes.	11
3.2	Composition of the forget set by code unit complexity and injected secret type. . .	12
3.3	Effect of mutation hardening on the functional correctness benchmarks. Test counts are means over the units, with standard deviations.	13
3.4	Character bigrams for the reference and generated text.	17
4.1	Unlearning hyperparameters. Values marked $\dagger$ follow the corresponding source implementation.	22
4.2	Evaluated unlearning techniques.	22
4.3	Configurations evaluated for RQ3.	22
5.1	Forget-Quality@10 (FQ) and UtilityEval pass@10 for the NPO variants under secret unlearning. Summary cells report the mean $\pm$ standard deviation across the three evaluated epochs and are green when the mean is at least 0.8 and red otherwise. .	25
5.2	Code unit unlearning after three epochs. All values are percentages. Green cells meet the 80% target, while red cells indicate near-zero performance ($\leq 10\%$). FQ denotes Forget-Quality@10, and Utility denotes UtilityEval pass@10. Higher values indicate better performance for both metrics.	30

5.3	Full-parameter GA relative to LoRA-based GA on Qwen.	35
1	Sensitivity of the Secret pass@10 best-epoch selection to a common hardened UtilityEval and Forget Quality threshold τ . Epoch agreement is calculated among configurations remaining eligible at τ , relative to the 0.80/0.80 selection. Frontier retention is relative to its four Pareto-optimal points.	47
2	One-factor-at-a-time sensitivity of the Secret pass@10 selection. One threshold is varied while the other is fixed at 0.80. Epoch agreement is relative to the selected epochs at 0.80/0.80 and is reported among eligible configurations. Frontier retention is relative to the four Pareto-optimal points at 0.80/0.80.	47
3	Sensitivity of the Secret pass@10 best-epoch selection to baseline-derived filtering at the fixed 0.80 hardened UtilityEval and Forget Quality thresholds. Epoch agreement is relative to the filtered selection. Frontier retention is relative to its four Pareto-optimal points.	48
4	Mean effects of alternative secret unlearning settings relative to default random ordering of forget and retain batches. Values report the technique mean $\pm$ sample standard deviation. Performance differences are averaged across both models and three epochs ($n = 6$ per technique). Positive values indicate improved performance.	49

List of Listings

3.1	Synthetic function used in the forget set. The highlighted line marks the injected PII (email address) located inside the function-level docstring.	12
3.2	Synthetic class used in the forget set. The highlighted line marks the injected password located inside the class body.	12
5.1	Progressive degradation of the raw generation over epochs on a UtilityEval task for Llama undergoing GA+KL unlearning.	23
5.2	PROD-unlearned generation from Qwen at epoch 1 (UtilityEval, code-unit setting; highlighted segment). The model generates a file-separator token, hallucinates an unrelated file path, resulting in an unexecutable solution. The task prompt is omitted for clarity.	30

Introduction

Recently, Large Language Models (LLMs) have become particularly prominent in software engineering tasks such as code generation, bug fixing, and automated testing [1–3]. Their effectiveness in these tasks primarily depends on large-scale raw code corpora, much of which originates from human-generated artifacts [4]. These corpora include publicly available software repositories, technical forums, and documentation [5, 6]. While such data helps align models with real-world development practices and provides natural examples, it also raises concerns about data security and privacy [7, 8], particularly given the presence of personally identifiable information (PII) and malicious code [9, 10]. Despite rigorous data-cleaning procedures, the complete removal of this PII and harmful code cannot be guaranteed. As a result, LLMs can still memorize and reproduce content that remains undetected in the training data, particularly issues that are recognized only after deployment [11, 12]. This issue may lead to the leakage of sensitive information, such as API keys and passwords, as well as the reproduction of copyrighted code or code containing cybersecurity vulnerabilities.

To mitigate the risk of LLMs generating harmful code, the influence of such data points should be removed or suppressed even after training is complete. The most straightforward way to remove this influence is to retrain the entire model from scratch while excluding the targeted data [13]. Although this approach provides strong privacy and security guarantees, repeating the full training process for every newly identified data instance is computationally prohibitive [14]. This becomes particularly unfeasible as the rapid adoption of LLMs, especially for code generation tasks, brings greater attention to issues surrounding data deletion. This challenge is further intensified by the possibility of deletion requests arriving sequentially after deployment [15]. Compliance with regulations such as the GDPR requires these requests to be addressed without undue delay, particularly under the *right to be forgotten*, which entitles individuals to request the erasure of their personal data [16–18]. As an alternative to costly model retraining, Machine Unlearning (MU) has been introduced as a mechanism that enables models to forget specific data points or subsets of data upon request [13, 19]. Unlike retraining from scratch, MU techniques aim to selectively remove the influence of targeted data from a trained model at lower cost while trying to achieve comparable removal effects [19]. These techniques use a *forget set* containing the data points to be removed and an optional *retain set* containing the data points to be preserved. Although MU techniques offer a computationally viable alternative to retraining, they rarely match the behavior of a reference model trained without the forget data. In practice, stronger forgetting tends to come at the cost of downstream performance [19–21]. Even though this trade-off has been studied primarily for natural text, its behavior in code generation capabilities remains comparatively underexplored, with only a handful of works addressing it directly [22–24]. This is critical because these techniques may not be directly applicable to code, which is more syntactically and semantically structured than natural text. Even when applicable, their effectiveness often depends on hyperparameter selection and the dataset, and there are no practical guidelines

for applying them effectively. This lack of guidance prevents informed decision-making and may negate the cost and resource advantages of MU over retraining.

To address this gap, this thesis asks one central question: how do the choice of unlearning technique and the construction of the forget and retain sets jointly shape the trade-off between forget quality, utility retention, and energy consumption? We answer this question by (1) analyzing the impact of various MU techniques, (2) examining how unlearning behavior differs across secrets and code units such as functions and classes, and (3) comparing how these trade-offs are affected by retain set size and the execution order of the forget and retain objectives. The contributions of this thesis can be summarized as follows. First, we introduce MOCHI (**M**achine **U**nlearning **o**f **C**ode with **H**idden **I**nformation), a benchmark for MU of code that comprises 600 synthetic functions and classes with injected secrets. Second, we shift the focus from evaluating unlearning algorithms in isolation to understanding how unlearning outcomes are shaped by the interaction among method choice, data selection, and the structure of the content to be removed. Third, we evaluate the transferability of MU techniques from secrets to complete code units and show the limitations of this transfer. Finally, we provide practical guidance to help practitioners make informed decisions when applying these techniques to code.

Related Work and Background

This section reviews prior work relevant to MU and introduces key concepts within the field.

2.1 Machine Unlearning

MU techniques aim to provide a superior trade-off between unlearning efficiency and effectiveness compared to retraining the model from scratch [13]. These methods are roughly categorized into two families: MU with parameter optimization and in-context unlearning [25]. In-context unlearning methods try to achieve the forgetting effect during inference without altering model parameters. This enables black-box unlearning, which can be applied even to closed-source models. These techniques are inherently resource-efficient since they do not require any additional training step. In contrast, parameter optimization-based methods achieve unlearning by updating model weights [25,26] and try to remove the knowledge from the model rather than masking it. Models are fine-tuned using carefully constructed counterexamples, replacement samples, or corrective updates targeting those specific instances. A key idea is to use the influence of unlearning points and minimize the impact these samples have on the model. This shifts the focus of unlearning from specific data instances to the features and labels through which data influences parameter updates. For example, Warnecke et al. [27] adopt this perspective by translating data effects into closed-form parameter updates. In this way, the undesired knowledge can be adjusted at the level of concepts or associations, rather than being restricted to individual samples. This allows for the precise removal of the undesired data’s influence without full retraining. However, its success depends on the completeness of the identified examples and the quality of the corrective data. In this work, we focus on parameter-optimization-based unlearning techniques for code generation tasks. These techniques can update either the entire model or a small subset of its parameters. While prior unlearning studies have explored both full-parameter updates [21,28] and parameter-efficient approaches [29,30], we use Low-Rank Adaptation (LoRA) [31] unlearning as our default setting. LoRA is widely used to train LLMs because it updates only a small number of additional parameters, thereby reducing memory, computation, and storage requirements [31]. This makes LoRA a practical choice for MU as LLMs continue to grow in size [32].

2.2 Machine Unlearning Objectives

MU techniques perform unlearning through a forget objective, optionally combined with a retain objective. The forget objective focuses on removing the influence of data in the *forget set*, which contains the samples targeted for unlearning. In contrast, the retain objective aims to

preserve model behavior on the *retain set*, containing samples that should remain unaffected by the unlearning process. In existing methods, these two objectives are often combined through a weighted sum of their respective losses:

$$\mathcal{L}_{\text{unlearn}} = \mathcal{L}_{\text{forget}}(\mathcal{D}_{\text{forget}}) + \lambda \mathcal{L}_{\text{retain}}(\mathcal{D}_{\text{retain}}),$$

where $\mathcal{D}_{\text{forget}}$ denotes the forget set, $\mathcal{D}_{\text{retain}}$ denotes the retain set, and λ controls the relative importance of the retain objective. A larger value of λ places greater emphasis on preserving retained behavior, whereas a smaller value places greater emphasis on the forget objective. We next introduce several forget and retain objectives used in MU.

2.2.1 Forget Objectives

Gradient Ascent (GA)

Gradient Ascent (GA) is a simple and widely adopted unlearning objective [33]. It performs gradient ascent on the prediction loss over samples in the forget set, which can be formulated as:

$$\mathcal{L}_{\text{GA}}(\Theta) = - \underbrace{\mathbb{E}_{(x_f, y_f) \sim \mathcal{D}_{\text{forget}}} [-\log(\pi_{\Theta}(y_f | x_f))]}_{\text{prediction loss on the forget set}} = \mathbb{E}_{(x_f, y_f) \sim \mathcal{D}_{\text{forget}}} [\log(\pi_{\Theta}(y_f | x_f))].$$

The objective reduces the log-likelihood of generating continuation y_f given input token sequence x_f . By negating the standard prediction loss, GA reverses the direction of conventional training updates on forget set samples. As a result, the model assigns lower probability to outputs associated with the data targeted for unlearning. Although GA is susceptible to catastrophic forgetting and can substantially degrade overall model utility, it remains a valuable baseline for comparing unlearning techniques [34].

Negative Preference Optimization (NPO)

Negative Preference Optimization (NPO) is an unlearning method inspired by direct preference optimization (DPO). In contrast to DPO, which relies on pairs of preferred and dispreferred samples, NPO operates using only the forget set. Its gradient can be expressed as

$$\nabla_{\Theta} \mathcal{L}_{\text{NPO}, \beta} = \mathbb{E}_{(x_f, y_f) \sim \mathcal{D}_{\text{forget}}} [W_{\Theta}(x_f, y_f) \nabla_{\Theta} \log \pi_{\Theta}(y_f | x_f)],$$

$$W_{\Theta}(x, y) = \frac{2\pi_{\Theta}^{\beta}(y | x)}{\pi_{\Theta}^{\beta}(y | x) + \pi_{\text{ref}}^{\beta}(y | x)}.$$

Here, $W_{\Theta}(x, y)$ adaptively down-weights forget set samples that have already been forgotten. The distribution π_{ref} represents the frozen reference (oracle) model, while π_{Θ} represents the current model undergoing unlearning. When the current model assigns a significantly lower probability to a forget set sample than the reference model, i.e., $\pi_{\Theta}(y | x) \ll \pi_{\text{ref}}(y | x)$, the corresponding weight becomes $W_{\Theta}(x, y) \ll 1$. Thus, the contribution of that sample to the gradient update is reduced relative to GA, resulting in a more stable unlearning process [34]. As $\beta \rightarrow 0$, the effect of the adaptive weighting diminishes, with $W_{\Theta}(x, y) \rightarrow 1$. Consequently, the NPO gradient approaches the GA gradient. By mitigating the catastrophic forgetting associated with GA, NPO is a useful point of comparison among forget objectives [34].

Probabilistic Redistribution for Output Distribution (PROD)

Probabilistic Redistribution for Output Distribution (PROD) is an unlearning technique designed for code unlearning tasks that operates at the token-distribution level [23]. For each forget set token, it removes the probability assigned to the targeted token. The remaining probability mass is then redistributed over the surviving vocabulary to construct a new distribution p_T from the model before unlearning. The model is then optimized to match p_T using the objective

$$\mathcal{L}_{\text{PROD}}(x_f, y_f; \pi_{\Theta}) = - \sum_{t=1}^L \sum_{w \in V} p_T(w \mid \cdot) \log \pi_{\Theta}(w \mid \cdot),$$

where V denotes the model vocabulary, L denotes the length of the forget sequence, and $\pi_{\Theta}(\cdot)$ denotes the output distribution of the model undergoing unlearning. By modifying the output distribution at the token level, PROD aims to suppress samples in forget set while limiting changes to the remaining model behavior. It has been shown to cause a smaller utility drop than NPO and GA while achieving comparable unlearning effects. PROD is therefore selected to investigate whether it outperforms other techniques for MU consistently.

2.2.2 Retain Objectives

Gradient Descent (GD)

Gradient Descent (GD) is the direct counterpart of GA. Rather than negating the prediction loss, GD minimizes the standard prediction loss over samples in the retain set. In other words, it follows the same formulation as GA, but with the sign reversed:

$$\mathcal{L}_{\text{GD}}(\Theta) = \underbrace{\mathbb{E}_{(x_r, y_r) \sim \mathcal{D}_{\text{retain}}} [-\log(\pi_{\Theta}(y_r \mid x_r))]}_{\text{prediction loss on the retain set}}.$$

It updates the model parameters by descending the gradient of this loss, thereby reinforcing the patterns present in the retained samples [35]. In this work, GD is selected as an oracle-free retain objective for preserving the generation capabilities of the model during unlearning.

Kullback–Leibler Divergence Minimization (KL)

Kullback Leibler Divergence Minimization (KL) encourages the unlearned model to preserve output distribution of the original model on the retain set $\mathcal{D}_{\text{retain}}$ [36], by minimizing the loss:

$$\mathcal{L}_{\text{KL}} = \frac{1}{|\mathcal{D}_{\text{retain}}|} \sum_{s \in \mathcal{D}_{\text{retain}}} \frac{1}{|s|} \sum_{i=2}^{|s|} D_{\text{KL}}(\pi_{\text{ref}}(s_{<i}) \parallel \pi_{\Theta}(s_{<i})).$$

Here π_{ref} denotes the frozen reference model and π_{Θ} the model undergoing unlearning. Given the prefix $s_{<i}$, each produces a probability distribution over the vocabulary for the next token. Minimizing the divergence between them keeps the unlearned model predictions on the retain set close to those of the reference model. Relative to GD, this requires an additional forward pass through the frozen model at every step. This introduces a trade-off between computational overhead and retention quality and it is a useful point of comparison among retain objectives.

2.3 Dataset Curation for Machine Unlearning

Dataset curation is a fundamental first step in MU tasks and directly shapes the unlearning process. Unlike standard training, where data is typically used for a single optimization objective, unlearning partitions data by role: the forget set contains elements targeted for removal. In contrast, the retain set contains elements whose behavior should be preserved [13]. Prior work selects unlearning targets either by identifying knowledge that models have memorized and can reproduce or by synthetically introducing it through fine-tuning before unlearning. Synthetic dataset curation with fine-tuning provides greater control and helps avoid contamination from pretraining data [21]. However, this scenario is also less realistic, creating a trade-off in dataset design. In the context of code unlearning, prior work has constructed forget sets from code files, deprecated APIs, code snippets vulnerable to cybersecurity attacks, and detected memorized secrets [23, 24, 28, 37]. These code artifacts and secrets are typically selected from existing datasets or scraped from public GitHub repositories.

Maini et al. introduced TOFU [21], one of the most widely used evaluation frameworks for natural language unlearning. TOFU consists of fictitious author profiles synthetically generated by GPT-4 [38], which are used to fine-tune the evaluated models. Because this knowledge is introduced only during fine-tuning and does not overlap with the pretraining data, TOFU provides a cleaner setting for evaluating unlearning. The fictitious authors are divided into forget and retain sets, together with an in-domain set containing real authors and general knowledge. Some prior work in code unlearning has attempted to follow this design by injecting PII into code snippets [22]. However, this scenario captures only one dimension of code unlearning and overlooks critical cases, such as the removal of copyrighted code. Motivated by this limitation, we introduce the MOCHI benchmark based on synthetic functions and classes with injected secrets. This enables evaluation beyond short-string memorization by incorporating semantically structured code units as unlearning targets.

2.4 Machine Unlearning for Code

A review of related work on MU for code generation tasks reveals only a limited body of research. In prior studies, GA [22–24] has been predominantly employed. These studies found that GA often causes a significant drop in utility, commonly referred to as catastrophic forgetting. To mitigate this issue, subsequent approaches incorporated retain objectives into the unlearning process, leading to the evaluation of techniques such as GA combined with GD [22] and GA combined with KL [22]. Other explored approaches include Direct Preference Optimization (DPO), where data to be forgotten is paired with desired behaviors [23], and NPO [23], which mitigates the catastrophic forgetting associated with GA by adaptively down-weighting forget set elements that have already been forgotten. Additional methods include PROD [23], a probability redistribution method that suppresses unwanted tokens. CodeEraser [24] further combines data filtering with optimization-based unlearning and applies GA, GD, and KL objectives to unlearn secrets while preserving surrounding code context.

Despite this methodological variety, two aspects of the unlearning process remain largely unexamined. First, existing studies treat the forget and retain sets as fixed inputs rather than as design choices. The order of forget and retain optimization and the composition of each set are not investigated, even though both can affect the forgetting–utility balance. Second, individual works target PII and hardcoded secrets within code [22, 24], as well as whole copyrighted files, vulnerable segments, and deprecated API calls [23]. However, no study contrasts secrets embedded in code with semantically coherent program units, such as functions and classes, under a common method and evaluation. This thesis addresses both gaps directly by treating set dynam-

ics and content type as central factors in unlearning behavior.

2.5 Evaluation of Machine Unlearning for Code

Existing work on code unlearning employs a broad range of metrics to evaluate both unlearning effectiveness and model utility. For utility evaluation, HumanEval [5] is among the most widely adopted benchmarks and has been used across numerous studies [22, 24, 28]. It includes self-contained Python functions designed to assess general coding ability. Functional correctness tests are also common for measuring the coding capabilities a model acquires through fine-tuning [22]. Beyond functionality-based evaluation, prior work also relies on similarity-based indicators of utility retention, such as BLEU [39], which captures word-sequence overlap between generated and reference code, and chrF [40], which measures character-level overlap, as well as direct measurements based on cross-entropy loss and perplexity over elements that should remain within the behavior of the model [22, 23, 37].

Unlearning effectiveness is frequently assessed using metrics that also appear in utility evaluation, including BLEU and perplexity. In addition, several studies adopt metrics that target the leakage of sensitive information in code snippets [22, 37]. The Min-k% membership inference attack [41] is commonly used to assess whether a given data point was included in the training data of a model. At the same time, other work evaluates robustness under adversarial conditions, such as prompt injection attacks, to assess whether unlearning techniques remain effective against malicious prompts [23, 28]. A further class of metrics quantifies memorization directly, including Memorization Accuracy (MA) [42] and Extraction Likelihood (EL) [43]. Following prior work, we combine similarity-based metrics with functional correctness tests. BLEU and chrF are used to assess similarity on the forget set, whereas functional correctness tests measure capabilities acquired through fine-tuning.

2.6 Resource Efficiency of Machine Unlearning Techniques

In the existing literature, the trade-off between forget quality and utility retention is the primary metric used to compare MU techniques, while energy efficiency is usually ignored. However, efficiency should also be a driving factor in selecting a suitable technique. Since not all sensitive data can be detected at once, this process is inherently continuous. Even though MU is significantly cheaper than retraining from scratch, its continuous nature can lead to gradually increasing costs [13]. To quantify this cost, previous work estimates the total training floating-point operations (FLOPs) using the formula ($6 \times \text{Total Training Tokens} \times \text{Parameter Size}$), or reports wall clock seconds, GPU hours, and memory [28, 30, 43–46]. Rather than relying solely on estimates and durations, in this thesis we use GPU energy consumption obtained with the CodeCarbon Python library [47]. Even though CodeCarbon is not definitive and can be imprecise in some cases, it is a lightweight solution that can be used out of the box and provides valuable comparison points across techniques run on similar hardware.

Methodology

This section describes the methodology used to evaluate MU for code generation tasks.

3.1 Research Questions

The main question of this thesis is what jointly shapes the trade-off among forget quality, utility retention, and energy consumption when applying MU in the context of code generation. To answer this question, we examine the effects of the unlearning technique, the type of content targeted for removal, and the design of the forget and retain sets with respect to their relative sizes and optimization order. Since a single experiment cannot isolate all of these factors at once, we decompose this question into three research questions:

- **RQ1: How are trade-offs between forget quality, utility retention, and energy consumption affected when machine unlearning techniques are applied to secrets in code?**

This research question aims to compare multiple MU techniques, such as GA [33], NPO [34], PROD [23], and their variants that utilize KL and GD retain objectives. Only a subset of these techniques have been applied to LLMs for code generation tasks, and only in a limited number of studies [22–24]. The objective is to examine how different forget objectives and the use of a retain set affect forgetting quality, code generation performance, and energy consumption. This analysis will help identify a suitable balance when applying MU to LLMs for code generation tasks.

- **RQ2: How does machine unlearning behavior differ between secrets and semantically structured code units?**

Most MU techniques have been developed and evaluated primarily for natural language text, with comparatively limited attention given to source code. It remains unclear whether these techniques transfer effectively to structured code units, which we define as complete functions or classes that form semantically and syntactically coherent components of a program. Existing work mainly focuses on removing arbitrary code snippets or secrets embedded in code, rather than entire code units [22–24]. Addressing this gap is critical because a function or class is a whole coherent semantic unit, and forgetting it may require extra attention and data processing steps. If existing techniques degrade or behave differently in this setting, their reported trade-offs may not hold for copyrighted code removal scenarios. Therefore, this research question tries to determine whether the success of MU techniques is transferable to whole functions and classes, and how the resulting trade-offs differ from those in the secret unlearning setting.

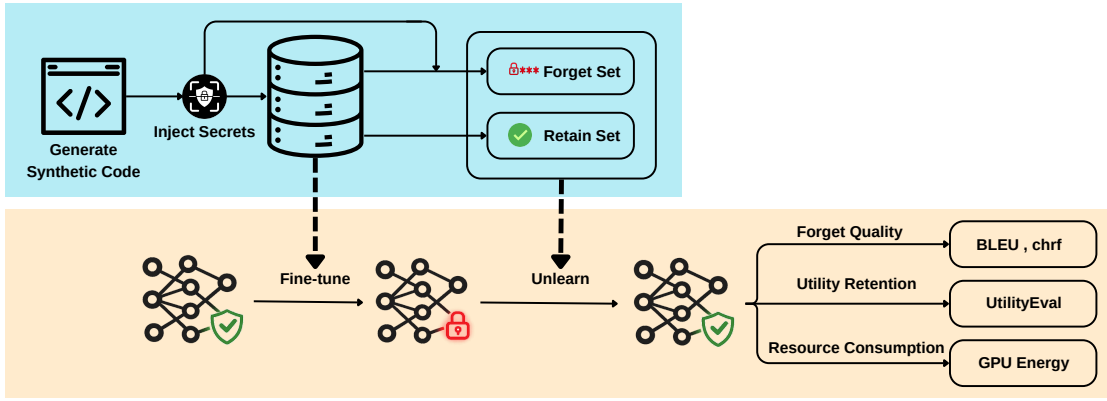


Figure 3.1: Overview of the experimental workflow. A pre-trained model is first fine-tuned on the synthetic dataset with injected secrets, then unlearned using the forget and retain sets. The resulting model is evaluated along three dimensions: forget quality, utility retention, and resource consumption.

- **RQ3: How do retain set size and forget–retain optimization order influence machine unlearning trade-offs for code generation tasks?**

The core components of unlearning are the objectives used to forget and retain, and the dataset design. This dataset is organized into forget and retain sets, which guide the model in removing unwanted elements while preserving desired behavior. However, design decisions surrounding these components, such as the size of the retain set and the execution order of the forget and retain objectives, are underexplored in existing work. To the best of our knowledge, no prior work in the literature examines how these characteristics affect MU tasks. The purpose of this research question is therefore to examine whether these aspects influence the overall unlearning process and can improve the trade-offs between unlearning effectiveness, utility retention, and energy consumption.

3.2 Study Design

In this section, we present the design of our study. The workflow is shown in Figure 3.1, where we fine-tune pre-trained LLMs on the synthetic dataset, apply each unlearning method under various configurations, and evaluate each resulting model for forget quality, utility retention, and resource consumption.

3.2.1 Benchmark Construction

To deepen our understanding of MU in code generation tasks, we introduce MOCHI, a benchmark focusing on Machine Unlearning of Code with Hidden Information. We create a novel dataset comprising functions and classes that are injected with secrets, such as API keys, passwords, and email addresses. Functions and classes were selected because they are widely used semantic units in code datasets [48–50]. Because the pre-training corpora of publicly released LLMs are typically unavailable [51], maintaining control over the knowledge a model acquires during fine-tuning by minimizing contamination with pre-training data is essential. Following

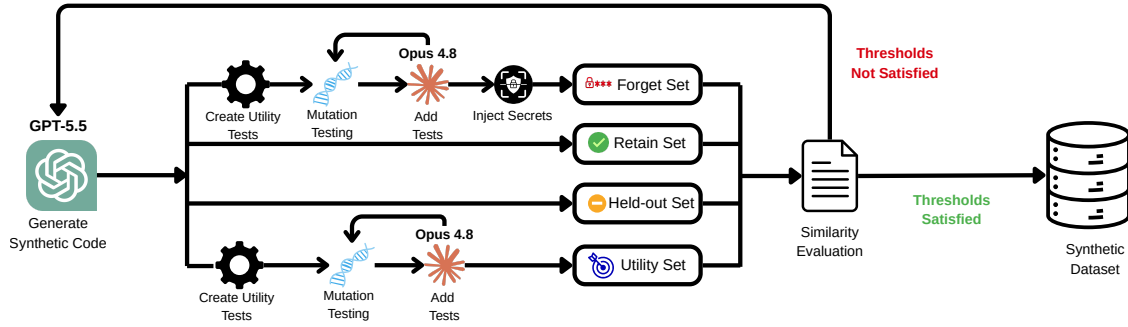


Figure 3.2: Overview of the benchmark construction workflow. GPT-5.5 generates candidate functions and classes, secrets are then planted into the forget set. All candidates pass through a similarity evaluation, elements that violate the configured similarity thresholds are regenerated.

TOFU [21] in its approach to benchmark construction for MU, we adopt a synthetic data generation process. We generate these synthetic code pieces by prompting GPT-5.5 [52] with various coding scenarios and randomizing all unit names so that the exact token sequences are highly unlikely to appear in any existing corpus. Then we fine-tune pre-trained models using this dataset as shown in Figure 3.1. As a sanity check, we evaluate the base models before fine-tuning and confirm that they do not confidently complete the planted secrets or code units. The primary objective of this benchmark is to assess both the effectiveness of unlearning and the retention of coding capabilities with respect to the fine-tuning dataset. Specifically, it evaluates whether the model reverts toward its baseline behavior on the forget set while preserving coding capabilities. This requirement is the driving motivation for using synthetically generated code. Figure 3.2 gives an overview of the benchmark construction workflow, from synthetic generation through similarity-based evaluations. We explain each stage in detail in the following subsections.

In detail, the dataset comprises four components: a forget set, a retain set, a utility set, and a held-out set, containing 150, 300, 100, and 50 code snippets, respectively. Each of these sets is divided equally between functions and classes (see Table 3.1). The forget and retain sets are the two driving sets for the unlearning and retention objectives. The utility set is used to measure the effect of unlearning on the utility acquired during fine-tuning. The held-out set contains functions and classes that do not contribute to the unlearning process.

Table 3.1: Overview of the dataset composition. Each component is divided equally between functions and classes.

Component	Functions	Classes	Total	Role
Forget set	75	75	150	Unlearning objective
Retain set	150	150	300	Retention objective
Utility set	50	50	100	Fine-tuning utility effect
Held-out set	25	25	50	In-domain, not unlearned or retained
Total	300	300	600	

Table 3.2: Composition of the forget set by code unit complexity and injected secret type.

Complexity	Functions	Classes	Total
Simple	25	25	50
Moderate	25	25	50
Complex	25	25	50
Total	75	75	150
Injected secrets: 50 API keys, 50 passwords, and 50 emails			

```
def synthetic_forget_function_UAKw9i(values):
    """Convert integers into sorted inclusive ranges.

    Duplicate values are ignored. Empty input returns
    an empty list. Each range is returned as a
    (start, end) tuple, where single values have
    equal ends.

    Example: synthetic_forget_function_UAKw9i(
        [4, 2, 3, 8]) -> [(2, 4), (8, 8)]
    email_EU1jjQ = "kxkdpi@example.com"
    """
    nums = sorted(set(values))
    if not nums:
        return []
    ranges = []
    start = nums[0]
    previous = nums[0]
    for value in nums[1:]:
        if value == previous + 1:
            previous = value
        else:
            ranges.append((start, previous))
            start = previous = value
    ranges.append((start, previous))
    return ranges
```

Listing 3.1: Synthetic function used in the forget set. The highlighted line marks the injected PII (email address) located inside the function-level docstring.

```
class synthetic_forget_class_aRdebu:
    """A chore wheel rotates through housemates while
    allowing skipped turns to be repaid later.

    Public methods:
        - skip_turn(person): mark the current person as
          owing a skipped turn.
        - next_housemate(): choose the next person,
          preferring anyone who owes a turn.

    Example:
        wheel = synthetic_forget_class_aRdebu(["Ada", "Ben"]);
        wheel.next_housemate() == "Ada"""
    def __init__(self, people):
        self.people = list(people)
        self.index = 0
        self.owed = []
        password_FrEHS2 = "DWY1gAACTP9gyv1p1BA"

    def skip_turn(self, person):
        if person not in self.people:
            return False
        if person not in self.owed:
            self.owed.append(person)
        return True

    def next_housemate(self):
        if self.owed:
            return self.owed.pop(0)
        if not self.people:
            return None
        person = self.people[self.index % len(self.people)]
        self.index += 1
        return person
```

Listing 3.2: Synthetic class used in the forget set. The highlighted line marks the injected password located inside the class body.

The forget set differs from other sets since we inject fake secrets and PII into functions and classes. These sensitive strings are placed either in the documentation immediately following the signature or as a variable assignment within the code. To answer **RQ2**, which analyzes the transferability of MU techniques from secrets to code units, the forget set varies across experimental settings. The secret forget set contains only the secret, whereas the code unit forget set contains the entire functions and classes in which these secrets appear. Examples of a forget function and

class are shown in Listings 3.1 and 3.2, respectively. They contain planted secrets: an email address representing PII in the docstring of the synthetic function, and a password in the body of the synthetic class.

Functions and classes are further categorized by complexity, computed via cyclomatic complexity (CC): (i) over the whole body for functions, and (ii) as the average CC across methods for classes. For classes, we exclude `__init__` methods, since in our synthetic code they handle variable assignment rather than business logic. We group code units into complexity bands as defined in McCabe’s software-quality metrics guidance [53,54]: simple (1–10), moderate (11–20), and complex (21–50). Sampling equally across these bands prevents the benchmark from being dominated by trivial examples. The resulting distribution is shown in Table 3.2.

Table 3.3: Effect of mutation hardening on the functional correctness benchmarks. Test counts are means over the units, with standard deviations.

Benchmark	Stage	Tests / Function	Tests / Class Method	Mutation score
ForgetEval	Initial	4.93 $\pm$ 2.40	3.23 $\pm$ 0.51	53.7%
	Hardened	16.61 $\pm$ 4.81	8.09 $\pm$ 3.06	79.4%
UtilityEval	Initial	8.40 $\pm$ 0.49	2.77 $\pm$ 0.23	79.2%
	Hardened	13.48 $\pm$ 3.57	12.72 $\pm$ 5.27	93.9%

Functional Correctness Tests

One of the critical trade-offs of MU techniques is utility degradation when trying to achieve high levels of forgetting. In the context of unlearning secrets injected into code and entire code units, code generation capabilities are the primary utility concern. To measure the downstream utility and collateral damage effects of unlearning, we use functional correctness tests targeting the functions and classes in the utility and forget sets. These unit-test suites are called UtilityEval and ForgetEval, respectively. We use UtilityEval as the utility measure rather than the HumanEval or MBPP general coding benchmarks [5,55]. This is because MOCHI targets samples memorized during fine-tuning, so the capabilities acquired at that step are the ones most at risk of degradation. Moreover, UtilityEval operates in the same domain as the forget and retain sets, making it a more meaningful utility measure than general-purpose benchmarks.

Prior work shows that LLMs can generate effective test suites [3,56]. We therefore generate the UtilityEval and ForgetEval functional correctness benchmarks using GPT-5.5. Since mutation analysis is considered one of the strongest indicators of test-suite adequacy [57,58], we use mutation testing to identify inadequate LLM-generated test suites. This process involves slightly altering the code to produce variants called mutants, and we check whether the tests detect the resulting change. A mutant is said to be killed if at least one test fails on it, indicating that the suite has sufficient coverage to detect that anomaly. We run mutation testing with `mutmut`¹, a mutation-testing tool for Python. We then provide the mutation report to Claude Opus 4.8², which generates additional tests. Using a different model for this hardening step reduces the likelihood that the additional tests reproduce the same weaknesses as the initial GPT-5.5 generated tests [59]. As reported in Table 3.3 this raises the mutation score from 53.7% to 79.4% for ForgetEval and from 79.2% to 93.9% for UtilityEval, together with the corresponding increase in test density. Finally, we revalidate that every solution passes its own tests.

¹<https://github.com/boxed/mutmut>

²<https://www.anthropic.com/news/claude-opus-4-8>

Code Sample Diversity and Similarity Evaluation

A key concern when constructing synthetic datasets with LLMs is ensuring the diversity and uniqueness of the generated samples. Insufficient diversity and excessive redundancy can reduce the downstream utility of the models and contaminate MU evaluations [21, 60, 61]. In particular, near-duplicates distributed across the forget and retain sets may cause the model to preserve and forget similar code simultaneously. Moreover, even when similar code samples belong to the same set, their individual forgetting or retention outcomes may overlap because of this similarity. To prevent such contamination and precisely evaluate unlearning effects, it is important to conduct a similarity evaluation on lexical, structural, and semantic dimensions. While lexical similarity measures the direct overlap of tokens or characters, structural similarity evaluates similarities in code organization, syntax, and data flow. Semantic similarity, in contrast, assesses whether code samples perform the same or related functionalities. Together, these three dimensions provide a more comprehensive understanding of similarity among code samples. Lexical similarity is measured using Jaccard overlap of the token IDs produced by the gpt-oss-120b³ tokenizer. Structural similarity is measured with CodeBLEU [62] under the default equal-weight configuration (n -gram, weighted n -gram, syntax, and dataflow matches, each = 0.25) [62]. Because CodeBLEU is asymmetric, we average the forward and reverse scores for each unordered pair. Semantic similarity is measured with code embedding models, validated across two independent models (SFR-Embedding-Code-400M_R⁴ and Qodo-Embed-1-1.5B⁵). These transformer-based embedding models can capture semantic representations of code samples in an embedding space. However, they are known to compress their outputs into a narrow region of that space, causing arbitrary pairs to appear similar [63]. This phenomenon is known as anisotropy. To mitigate this limitation, we mean-center the embeddings before comparison by subtracting the mean embedding vector from each embedding, thereby shifting the center of the overall distribution to the origin [64].

Across the whole dataset, we treat 0.6 as a maximum similarity acceptance threshold for all three metrics and rewrite or regenerate any code that violates it. This is evaluated over all 179,700 pairs of the 600 samples. For semantic (embedding) similarity, 0.6 is an established cosine-similarity threshold for flagging content that is related but not identical [65]. Recent calibration of cosine similarity against empirical semantic-duplicate rates further places the decision boundary in the 0.5–0.6 range [66]. For token similarity, large-scale code-deduplication pipelines typically use a looser threshold of 0.7 [67, 68], so our 0.6 cut is stricter and aims to bias construction toward well-separated examples. After applying these filtering steps, the evaluation reveals that no pair exceeds 0.6 on any of the three similarity dimensions. As can be seen in Figure 3.3, CodeBLEU and token Jaccard similarity both concentrate on low values. The most similar pairs reach only 0.55 CodeBLEU score and 0.53 token Jaccard similarity. The structural dimension is most evident in the closest CodeBLEU pair, where two synthetic classes show strong syntactic similarity (0.69) and dataflow similarity (0.83), despite weak n -gram overlap (0.34). This suggests that CodeBLEU captures structurally similar code that lexical overlap alone would likely treat as irrelevant. We therefore apply the threshold to the aggregate CodeBLEU score rather than to each component. On the semantic axis, the mean-centered cosine is concentrated near zero and reaches the 0.6 threshold for no pair under either model. As shown in Figure 3.4, the contrast with the raw cosine quantifies the anisotropy directly. Under raw cosine, the mean pairwise similarity is 0.57–0.59, and between 28% (Qodo) and 40% (SFR) of all pairs exceed 0.6, whereas after mean-centering, that fraction is exactly zero.

³<https://huggingface.co/openai/gpt-oss-120b>

⁴https://huggingface.co/Salesforce/SFR-Embedding-Code-400M_R

⁵<https://huggingface.co/Qodo/Qodo-Embed-1-1.5B>

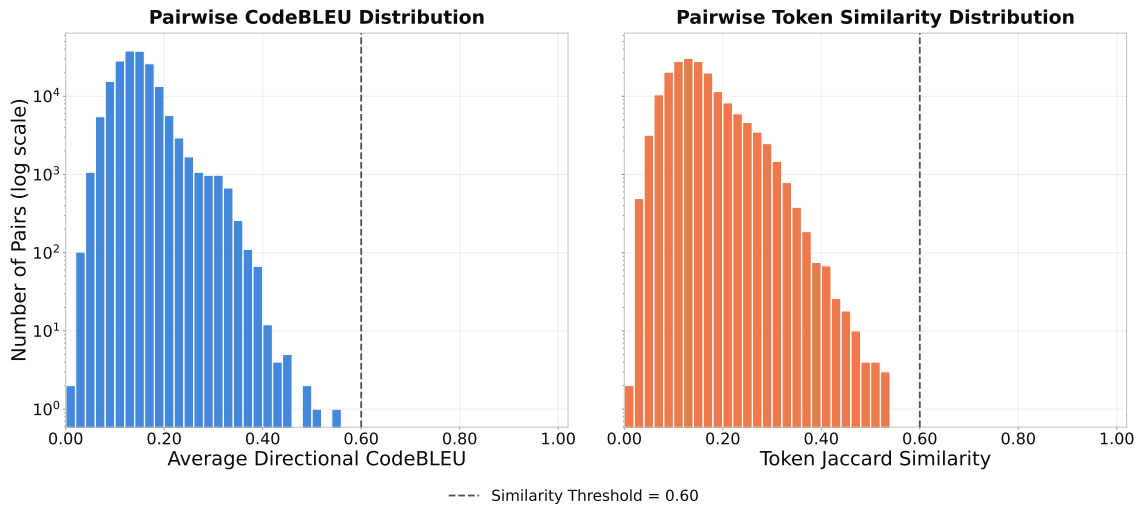


Figure 3.3: Pairwise lexical and structural similarity over all 179,700 pairs of the final dataset (log-scaled counts). Left: direction-averaged CodeBLEU; Right: token Jaccard similarity. Both lie entirely below the 0.6 threshold (dashed red), with maxima of 0.55 and 0.53 respectively

3.2.2 Model Selection

Evaluating different LLM architectures is important for understanding how differences in training objectives, data scope, and architecture affect the trade-offs of MU techniques. Therefore, we select two comparably sized models from different modern families: the code-specialized Qwen-2.5-Coder-3B (Qwen)⁶ [69] and the general-purpose Meta Llama 3.2 3B (Llama)⁷ [70].

3.2.3 Machine Unlearning Objective Selection

We evaluate GA, NPO, and PROD forget objectives, and GD and KL retain objectives. GA is included as a commonly used baseline, NPO as an approach designed to mitigate the instability associated with GA, and PROD as a method previously proposed specifically for unlearning code. GD and KL are selected as representative retain objectives, with GD reinforcing the retain data directly and KL preserving the original output distribution of the model.

3.2.4 Evaluation

Forgetting Evaluation via Suffix Reconstruction

We evaluate forgetting by testing whether the model can still reconstruct the targeted content, following the prefix-suffix reconstruction game of Carlini et al. [71]. The model is given a prefix p from the training data and is prompted to generate the corresponding suffix s . For code units, the prefix consists of the function or class signature together with its documentation, while the suffix corresponds to the code unit body. In contrast, in secret unlearning, the prefix is the code snippet that precedes the secret, and the suffix corresponds directly to the secret value.

⁶<https://huggingface.co/Qwen/Qwen2.5-Coder-3B>

⁷<https://huggingface.co/meta-llama/Llama-3.2-3B>

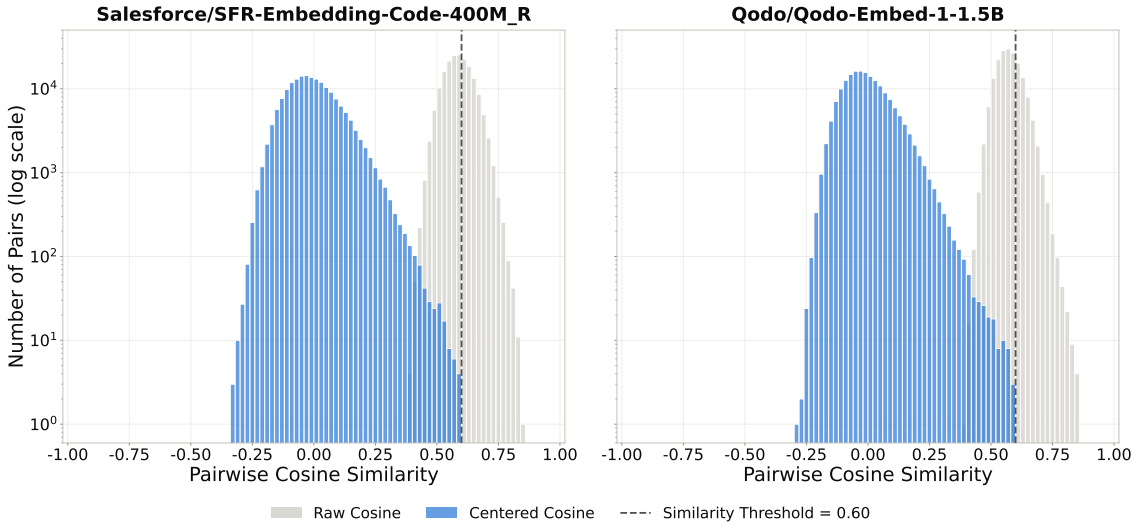


Figure 3.4: Pairwise cosine similarity over all 179,700 pairs for two code embedding models. Raw cosine (grey) is compressed into a narrow band near 0.6, illustrating embedding anisotropy. After mean-centering (blue), the distribution re-centers near 0.

Metrics and Tools

Forget quality. To quantify forget quality for secrets and copyrighted code, we first define the goal of unlearning in each case. For secrets, revealing more characters reduces the uncertainty of an attacker and can make attacks easier by lowering the number of guesses required. For instance, prior work on partial-password authentication shows that partial observations can increase the success of attacks [72]. The goal is therefore to minimize overlap between the generated output and the original secret. We measure recovery at the character-level using *chrF* [40], which captures how much of the secret reappears in the output by using an F-score based on character n-grams. The formula for the chrF score is:

$$\text{chrF} = \frac{(1 + \beta^2) \text{chrP} \cdot \text{chrR}}{\beta^2 \text{chrP} + \text{chrR}}$$

where chrP is the precision that denotes the percentage of character n-grams in the generated output that also occur in the reference text, while chrR is the recall that denotes the percentage of character n-grams in the reference text that are also present in the generated output. β is used for allocating more importance to chrR, when $\beta = 1$ they are weighted equally. An example can be seen in Table 3.4, where character bigrams are used. For the reference text **password**, there are seven bigrams, while the generated output **pass** contains three. All three generated bigrams also occur in the reference, yielding a precision of $3/3 = 1.0$. Three of the seven reference bigrams are recovered. The recall is therefore $3/7 \approx 0.43$. With $\beta = 1$, this results in a chrF score of approximately 0.60. This score ranges from 0 to 1, with higher values indicating greater character-level overlap between the generated and reference texts.

For copyrighted code, intellectual property rights protect expression rather than ideas [73,74]. This makes textual similarity a relevant signal for copyright-oriented evaluation, since literal reproduction of source code is more directly related to copyright concerns than functional equivalence alone. Following this, we use a word-level similarity metric, more specifically *BLEU* score [39], which is also aligned with previous code unlearning work [23].

Table 3.4: Character bigrams for the reference and generated text.

Text	Character bigrams
Reference: password	pa, as, ss, sw, wo, or, rd
Generated: pass	pa, as, ss

In contrast to chrF, BLEU score uses word-level n -grams instead of character-level n -grams, which is more suitable for code blocks with formula:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad \text{BP} = \begin{cases} 1, & c > r \\ \exp \left(1 - \frac{r}{c} \right), & c \leq r, \end{cases}$$

where p_n represents the precision for n -grams of order n , and w_n determines the weight of each n -gram order. The brevity penalty BP reduces the score when the generated text is shorter than the reference text, where c is the length of the generated text and r is the length of the reference text. Similar to chrF, BLEU score is also bounded between 0 and 1, with values closer to 1 indicating higher levels of similarity between two blocks of text. Both scores are computed with sacreBLEU using its default configuration [75].

Inspired by $\text{pass}@k$, which measures test success across k generations, we introduce $\text{Similarity}@k$. This metric reports the highest similarity score obtained across k independently generated outputs, where the similarity score corresponds to BLEU for generated code and chrF for generated secrets. We mainly report forget quality ($\text{Forget-Quality}@k$), defined as the complement of $\text{Similarity}@k$:

$$\text{Similarity}@k = \max_{i=1, \dots, k} s(y_i, y_{\text{ref}}) \quad \text{Forget-Quality}@k = 1 - \text{Similarity}@k.$$

Here, s denotes the similarity metric, y_i is the i -th generated output, and y_{ref} is the reference text. Higher $\text{Similarity}@k$ indicates greater recovery of the forgotten content, whereas higher $\text{Forget-Quality}@k$ indicates better forgetting.

Utility retention. Utility retention is evaluated based on coding capabilities acquired during the fine-tuning process. To this end, we utilize the functional correctness tests `UtilityEval` and `ForgetEval`, which target the utility of code samples that do not participate in the unlearning process and the utility of samples that are closely related to forgotten elements, respectively. We report $\text{pass}@k$ as the utility score, which is the fraction of tasks with at least one passing solution among k generations.

Resource consumption. We use GPU energy consumption as the primary measure of resource consumption for MU techniques, which is estimated using the `CodeCarbon` Python library [47].

Experimental Design

In this section, we describe our experimental design decisions and configurations for the unlearning process.

4.1 Fine-Tuning

As an initial prerequisite, we fine-tuned the selected models using LoRA [31] on the synthetic dataset generated for the unlearning study. To induce overfitting and encourage strong memorization, as also pursued in the TOFU benchmark [21], we replicated each sample so that it appears three times in the training set. We trained the models on this extended dataset for 5 epochs using a learning rate of 0.0002 , a standard setting for LoRA fine-tuning [31] and an effective batch size of 32. We then measured the average similarity between the generated outputs and the corresponding training data for both code units and secrets to verify memorization. This analysis was performed for both the base models and each epoch of the fine-tuned models. Figure 4.1 shows the resulting memorization trends across training. We use the checkpoint from the fourth epoch for the unlearning experiments, as it is the earliest checkpoint at which both models reach a memorization plateau for both code units and secrets.

Before unlearning, we filter the evaluation data based on what the fine-tuned models have memorized. We keep only the secrets and code units that the models reproduce exactly in at least one of ten generations, and only the test tasks that they pass in at least one of ten attempts. As shown in Figure 4.2, both models exactly reproduce a large fraction of secrets (86.0% for Llama, 90.0% for Qwen). The weakest memorization occurs in the code unit forget set on Llama, where exact reproduction covers just 97 of 150 samples (64.7%), compared to at least 89.0% for all other code unit sets. Even in this worst case, the filtered set remains large enough to support reliable forget quality evaluation. On the functional side, both models pass 83.3% of ForgetEval tasks and almost all UtilityEval tasks (at least 99.0%). This filtering ensures that every metric starts from a baseline of 1.0, so that subsequent results capture the relative utility degradation and forget quality after unlearning without contamination from incomplete baseline memorization or test coverage.

4.2 Unlearning

Once we fine-tune models to create a baseline that memorized the synthetic dataset, we conduct MU. Since unlearning is itself a training procedure, the learning rate is a critical hyperparameter that directly affects the success of forgetting, and its effect can vary substantially across forgetting

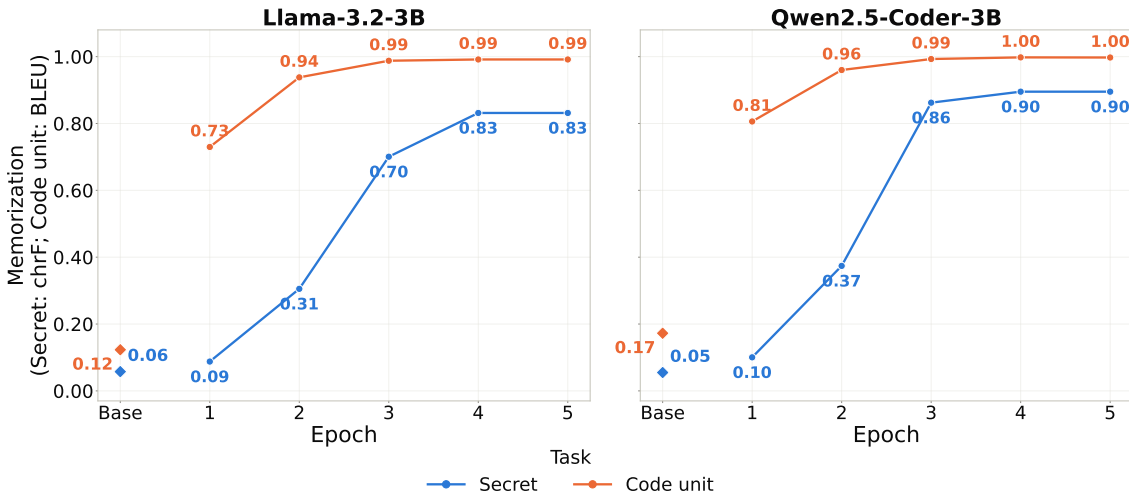


Figure 4.1: Mean generated-target similarity across the base and five training epochs for both models. Secret memorization is measured with chrF and code unit memorization with BLEU, averaged over 150 examples.

objectives. We therefore perform a grid search on Qwen over the three forgetting objectives GA, NPO, and PROD. Each objective is run and evaluated for three epochs. For GA and NPO, we search over learning rates $\{1e-6, 1e-5, 5e-5, 1e-4\}$. Exploratory analysis revealed that PROD requires higher learning rates to achieve comparable forgetting, we therefore shift the search space toward higher values and drop $1e-6, 1e-5$ and add $3e-4, 4e-4, 5e-4$ ($\{5e-5, 1e-4, 3e-4, 4e-4, 5e-4\}$).

We define an optimal region where both forget quality and utility retention are at least 0.8, and select the learning rates for which the most epochs fall in this region. As presented in Figure 4.3, for GA, $5e-5$ is selected with two epochs in the optimal region. For NPO and PROD, $1e-4$ and $3e-4$ are selected, respectively, since all epochs satisfy the criterion. The selected unlearning hyperparameters across all setups are summarized in Table 4.1. Unlearning is applied directly to the LoRA adapters obtained during fine-tuning. Since memorization was induced through these adapters, updating only the adapters is sufficient and computationally cheaper than updating the full parameters. We compare these two update scopes in an ablation study in Section 5.4. All unlearning experiments were conducted on a single NVIDIA A100 GPU with 80 GB of memory.

4.2.1 Experimental Configurations

To answer our research questions systematically, we design multiple experimental configurations and organize them according to the research question they address:

- **RQ1: Trade-offs of unlearning techniques for secrets.** As discussed in the previous sections, we use NPO, GA, and PROD as forget objectives, and KL and GD as retain objectives, combining them exhaustively. As shown in Table 4.2, this gives three forget-only methods and six combined methods, for a total of nine experimental setups. We use secret unlearning for this research question.
- **RQ2: Secret vs. code unit unlearning.** To answer this research question, we apply the same unlearning techniques in the code unit unlearning setup and compare the results to assess the transferability of unlearning techniques to semantically structured code units. By

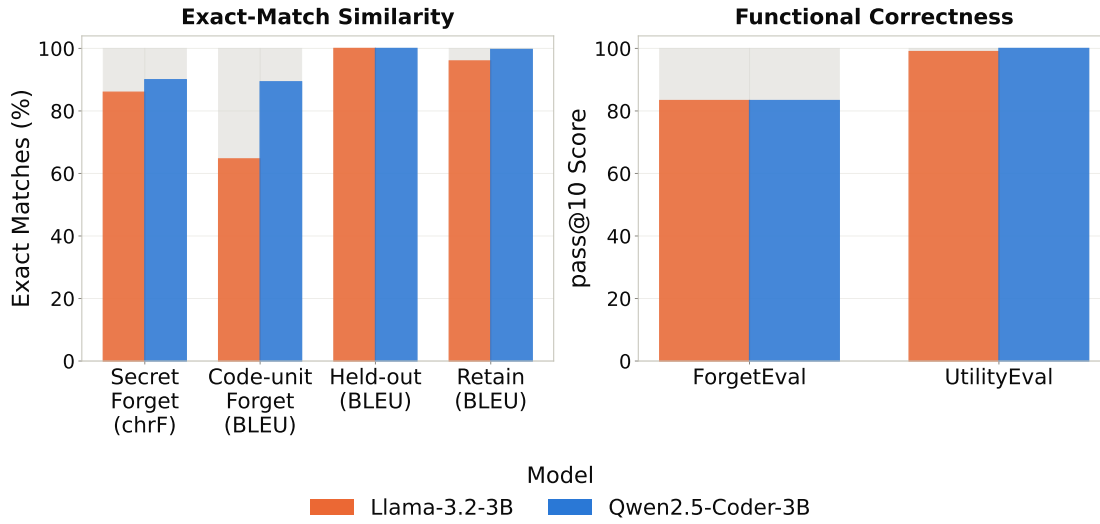


Figure 4.2: Performance of the fine-tuned models at checkpoint four before unlearning. The left panel shows the percentage of items with at least one exact match among ten generations, while the right reports ForgetEval and UtilityEval pass@10 scores.

varying only the forget set while holding all other parameters fixed, we isolate the effect of what is being unlearned.

- RQ3: Retain set size and objective ordering.** To analyze the effects of retain and forget objective dynamics on MU trade-offs, we vary several characteristics of the experimental configuration across two dimensions: (i) the ordering strategy between forgetting and retain optimization, and (ii) the relative size of the retain set compared to the forget set. The resulting experimental configurations are summarized in Table 4.3. Rather than exhaustively evaluating the Cartesian product of all dimensions, we adopt a controlled ablation design. In each experiment, we vary only one dimension at a time while keeping the remaining dimensions fixed at their default values. This design enables a more direct interpretation of how each factor affects the trade-off between forget quality and utility preservation. Unless stated otherwise, the default configuration uses mixed forget–retain objective ordering and a retain set equal in size to the forget set.

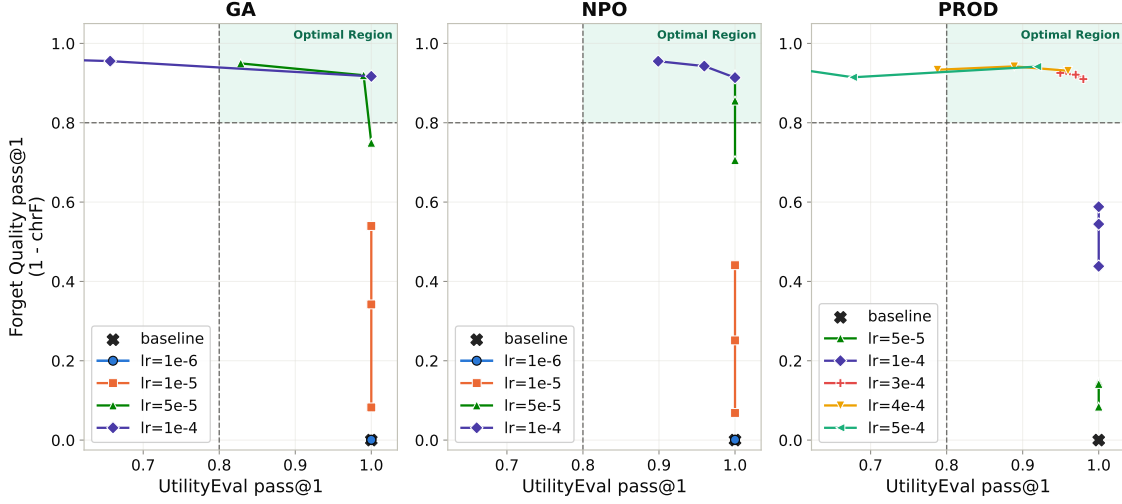


Figure 4.3: Trade-off between forgetting quality and model utility across learning rates for GA, NPO, and PROD. Each line corresponds to a learning rate, with points representing different epochs. Higher values on both axes are preferred, making the upper-right region the most favorable area.

Table 4.1: Unlearning hyperparameters. Values marked $\dagger$ follow the corresponding source implementation.

Setting	Value
<i>Shared configuration</i>	
Oracle model	Frozen 8-bit quantized model
Optimizer	AdamW
Effective batch size	32
Precision	bf16
Unlearning epochs	3
<i>Forget objectives</i>	
GA learning rate	5e-5
NPO learning rate	1e-4
NPO β	0.1 †
PROD learning rate	3e-4
PROD α	0 †
PROD p	0.8 †
<i>Retain objectives</i>	
Retain weight λ_{ret}	1.0

Table 4.2: Evaluated unlearning techniques.

Forget objective	Retain objective
GA	–
	GD KL
NPO	–
	GD KL
PROD	–
	GD KL

Table 4.3: Configurations evaluated for RQ3.

Dimension	Configuration
Ordering	Forget $\rightarrow$ Retain
	Retain $\rightarrow$ Forget
	Random
Retain set size	Equal to forget set
	Double the forget set

Results and Discussion

This section presents and analyzes the experimental results to answer the three research questions.

5.1 RQ1: Machine Unlearning Technique Trade-offs for Secrets

RQ1 investigates how applying GA, NPO, and PROD, optionally combined with retain objectives such as GD and KL, affects forget quality, utility retention, and resource consumption. To answer this question, we evaluate each technique on the task of unlearning secrets embedded in functions and classes using the MOCHI benchmark.

GA forget objective Our results suggest that MU techniques using GA exhibit a clear trade-off between utility retention and forget quality. As shown in Figure 5.1, for both models, coding capabilities tend to decrease as unlearning progresses, while forget quality increases across epochs. Listing 5.1 provides a qualitative example of this degradation observed in the generated code. After the first epoch of unlearning, the generated code still passes the utility test. By the second epoch, it remains syntactically correct, but the underlying logic is broken. By the third epoch, the implementation becomes invalid and can no longer be executed.

Plain GA preserves utility considerably better than its retain objective variants across the evaluated epochs. This advantage persists through the final epoch, where at least 69.7% of baseline utility is retained across both models. Interestingly, adding a retain objective to GA accelerates

```
# Epoch 1 -- functionally correct
for bracket_kwh, bracket_rate in energy_brackets:
    ...
    remaining -= used_here ← state updated

# Epoch 2 -- syntactically valid, semantically
incorrect
for bracket_kwh, bracket_rate in energy_brackets:
    ...
    remaining - used_here ← result discarded

# Epoch 3 -- syntactically invalid
remaining = max(k ← generation ends here
```

Listing 5.1: Progressive degradation of the raw generation over epochs on a UtilityEval task for Llama undergoing GA+KL unlearning.

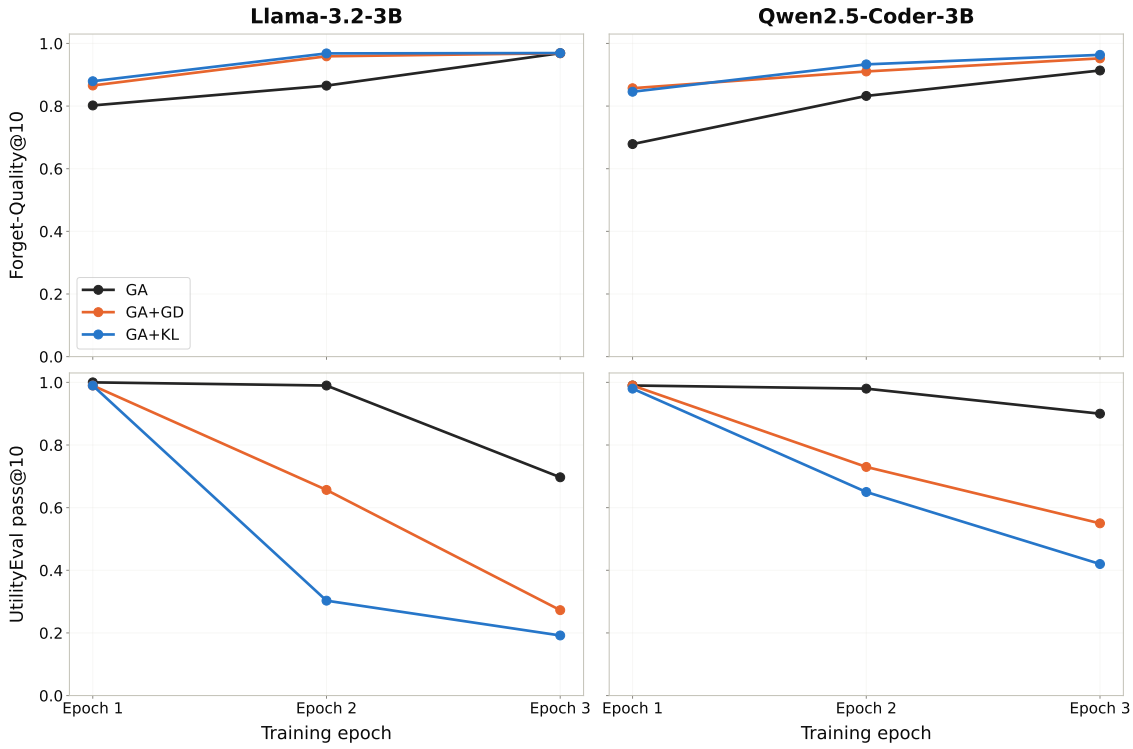


Figure 5.1: Epoch-wise performance of GA variants under secret unlearning. Columns correspond to the different models used in the experiments. Rows report Forget-Quality@10 and UtilityEval pass@10. Higher values indicate better performance for both metrics.

utility collapse at earlier epochs. For instance, at epoch 2, plain GA retains 99.0% of UtilityEval performance on Llama and 98.0% on Qwen, whereas GA+KL retains only 30.3% and 65.0%, respectively. By epoch 3, the utility of the retain objective variants drops to as low as 19.2%. Notably, these large differences in utility are accompanied by only minor differences in forget quality. Thus, GA+GD and GA+KL are competitive only early in the unlearning process. Their most favorable trade-offs, determined by equally weighting utility retention and forget quality, occur at epoch 1 on both models. At these points, they achieve forget quality between 84.6% and 87.9% while retaining between 98.0% and 99.0% of utility. Plain GA deteriorates more steadily: its most favorable trade-off occurs at epoch 2 on Llama, where it achieves 86.5% forget quality and 99.0% utility, and at epoch 3 on Qwen, where it achieves 91.4% forget quality and 90.0% utility.

Takeaway: GA without a retain objective provides a more stable utility–forgetting trade-off, while retain steps may be beneficial at early epochs but cause substantial degradation later.

NPO forget objective. In contrast to GA, NPO provides more reliable and stable utility retention while achieving similar forget quality. This behavior is consistent with the stabilizing effect of adaptive weighting introduced with NPO. This stability is evident in Table 5.1, which shows that, except for the plain NPO configuration on Llama, all NPO variants retain at least 93% utility at every epoch and average at least 97% utility across epochs.

Combining NPO with a retain objective further improves utility retention, with a maximum drop of only 2.0% across all epochs. The two retain objectives behave similarly on Llama, where the GD and KL variants yield nearly identical trade-offs, achieving 89–91% forget quality while retaining 98–100% utility. On Qwen, the KL variant achieves clearly higher forget quality than GD (86–87% vs. 75–80%) while maintaining nearly identical utility retention. Plain NPO can also provide a balanced trade-off, but its behavior is less consistent across the two models.

Table 5.1: Forget-Quality@10 (FQ) and UtilityEval pass@10 for the NPO variants under secret unlearning. Summary cells report the mean $\pm$ standard deviation across the three evaluated epochs and are green when the mean is at least 0.8 and red otherwise.

Model	Variant	FQ e1→e2→e3	FQ mean $\pm$ SD	UtilityEval e1→e2→e3	UtilityEval mean $\pm$ SD
Llama	NPO	0.88→0.94→0.99	0.94 $\pm$ 0.05	1.00→0.67→0.25	0.64 $\pm$ 0.37
	NPO+GD	0.90→0.91→0.90	0.90 $\pm$ 0.00	0.98→1.00→1.00	0.99 $\pm$ 0.01
	NPO+KL	0.89→0.91→0.89	0.90 $\pm$ 0.01	1.00→1.00→1.00	1.00 $\pm$ 0.00
Qwen	NPO	0.84→0.91→0.92	0.89 $\pm$ 0.04	0.99→0.98→0.93	0.97 $\pm$ 0.03
	NPO+GD	0.75→0.76→0.80	0.77 $\pm$ 0.03	1.00→0.98→0.99	0.99 $\pm$ 0.01
	NPO+KL	0.87→0.86→0.86	0.86 $\pm$ 0.01	0.99→0.99→0.99	0.99 $\pm$ 0.00

Takeaway: NPO preserves coding capabilities well across epochs, particularly when combined with a retain objective. NPO+KL provides the most consistently strong utility–forgetting trade-off across both models.

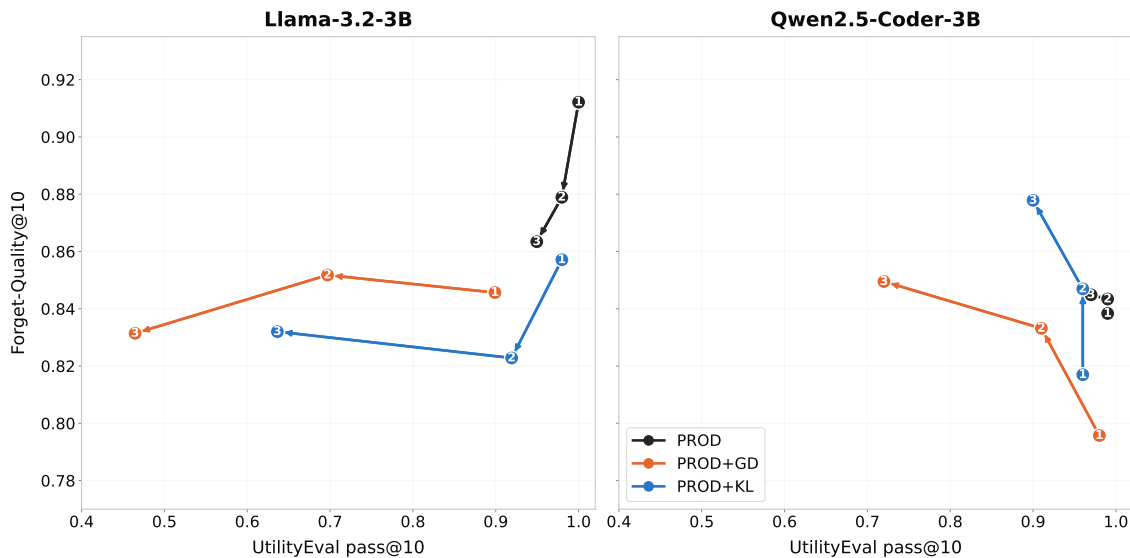


Figure 5.2: Epoch-wise utility–forgetting trajectories for PROD variants under secret unlearning. The axes report UtilityEval pass@10 and Forget-Quality@10, with higher values preferred. Point labels denote epochs and arrows indicate training direction.

PROD forget objective. In terms of utility preservation stability, PROD lies between GA and NPO. As shown in Figure 5.2, plain PROD consistently sustains more utility than its retain objective variants at the same epochs. In particular, plain PROD preserves at least 94.9% of baseline utility while achieving at least 83.8% forget quality, indicating behavior closer to NPO in terms of stability across epochs. However, adding a retain objective makes utility retention substantially less stable, which resembles the behavior observed for GA. By epoch 3, PROD+GD retains only 46.5% of baseline utility on Llama and 72.0% on Qwen, corresponding to reductions of 48.5 and 25.0 percentage points, respectively, relative to plain PROD. As in the NPO family, the KL retain objective provides a consistently more favorable trade-off than GD for PROD. PROD+KL Pareto-dominates PROD+GD in four of the six matched model–epoch comparisons and retains more utility in five of six. At epochs 2 and 3, its utility advantage ranges from 5.0 to 22.2 percentage points, while the forget quality of the two variants differs by at most 2.9 percentage points.

Takeaway: Plain PROD provides stable utility retention across epochs; however, adding a retain objective reduces this stability. Among retain variants, PROD+KL mostly provides a better utility–forgetting trade-off than PROD+GD.

Cross-technique comparison. To compare MU techniques, we select the epoch that maximizes the equally weighted mean of Forget-Quality@10 and UtilityEval pass@10 for each model–technique combination. We require both scores to be at least 0.8 out of 1.0. This isolates the region in which forgetting is effective while utility is largely preserved. As shown in Figure 5.3, the technique that provides the most favorable utility–forgetting trade-off depends on the model being unlearned. Nevertheless, NPO-based techniques generally occupy the high-utility region while achieving competitive forget quality, with NPO+KL emerging as one of the strongest balanced options across models. By contrast, adding a retain objective to GA or PROD does not consistently improve the trade-off and, in some cases, produces a less favorable balance than the corresponding plain forget objective.

For Llama, plain PROD at epoch 1 achieves the strongest trade-off, combining 91.2% forget quality with unchanged utility, and Pareto-dominates every other selected configuration. It is closely followed by NPO+GD and NPO+KL, both selected at epoch 2 and combining approximately 91% forget quality with full utility retention. For Qwen, the Pareto frontier contains three points. By achieving 91.1% forget quality and 98.0% utility, plain NPO at epoch 2 provides the most balanced point. Relative to this point, plain GA at epoch 3 improves forget quality by only 0.2 percentage points while sacrificing 8.0 percentage points of utility. Conversely, NPO+KL at epoch 1 gains 1.0 percentage point of utility at the cost of 4.2 percentage points of forget quality. Thus, Llama has a single dominant configuration, whereas Qwen presents a choice between a balanced solution and small shifts toward either stronger forgetting or greater utility retention.

We assess the robustness of the best epoch selection by varying the UtilityEval pass@10 and Forget-Quality@10 thresholds jointly and independently (Appendix Tables 1 and 2). All 18 configurations (9 techniques $\times$ 2 models) remain eligible across thresholds of 0.65 and 0.80, with 17 of 18 selected epochs unchanged and all 4 Pareto points retained. Coverage decreases primarily when the forget quality threshold exceeds 0.80 which supports the 0.80 threshold decision as a balanced cutoff. We also repeat the selection without baseline exact memorization and UtilityEval pass filtering (Appendix Table 3). Filtering changes the number of qualifying checkpoints only from 36 to 35, preserves all 18 configurations, and leaves 15 selected epochs and three of four Pareto points unchanged. Overall, the main findings are robust to both threshold choice and filtering.

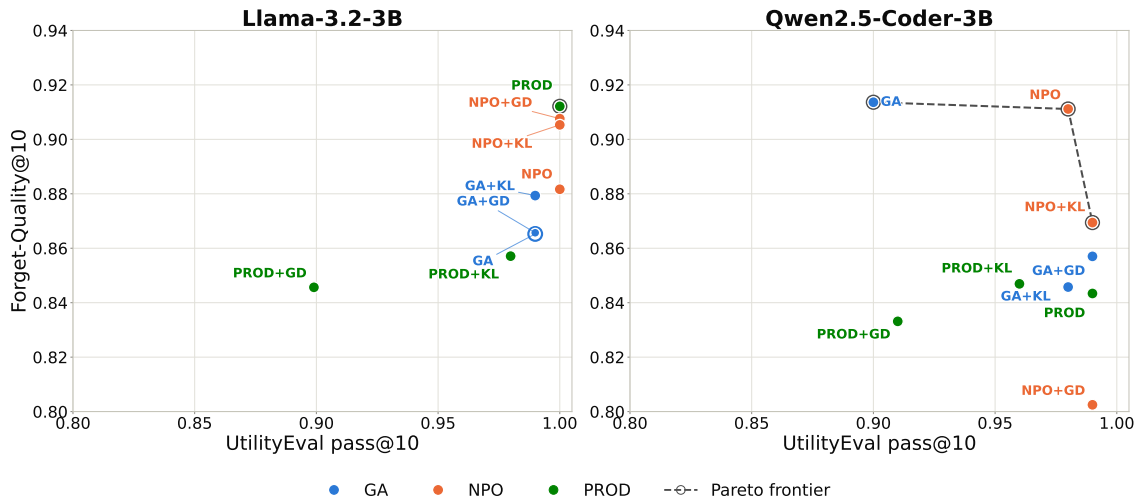


Figure 5.3: Utility–forgetting trade-off for the selected secret setting checkpoints. For each model–technique configuration, we consider epochs satisfying the minimum thresholds of 0.80 for both UtilityEval pass@10 and Forget-Quality@10 and select the epoch maximizing their equally weighted mean.

Takeaway: The most favorable utility–forgetting trade-off depends on both the model and the point at which unlearning is stopped. Across models, NPO+KL emerges as a safe and consistently strong option that combines high utility retention with competitive forget quality.

GPU energy consumption. We compare the GPU energy consumption of MU techniques across both models by reporting the mean with half the range between the two model measurements. As shown in Figure 5.4, the standalone forget objectives consume less energy than their retain objective variants in every forget family. GA is the least energy-intensive standalone objective. It consumes 0.424 ± 0.030 Wh, compared with 0.721 ± 0.067 Wh for NPO and 0.732 ± 0.070 Wh for PROD. Overall, NPO and PROD exhibit very similar GPU energy consumption across their corresponding standalone and retain-objective variants. Furthermore, the KL retain objective consistently consumes more energy than GD for every forget family and on both models. Relative to GD, KL increases GPU energy consumption by $43.7 \pm 2.3\%$ for NPO, $44.4 \pm 1.0\%$ for PROD, and $55.2 \pm 3.7\%$ for GA. This additional cost is likely due to KL requiring forward passes through a reference model. Although all of these measurements are negligible compared to retraining a model from scratch, the differences between techniques may become substantial as model sizes increase and unlearning operations are performed repeatedly.

Takeaway: GA consumes less GPU energy than NPO and PROD. Forget-only methods are also less energy-demanding than variants with retain objectives, with KL consistently requiring more energy than GD.

Utility around secrets. MU techniques aim to prevent models from leaking sensitive information without affecting the functionality surrounding the target. To test whether unlearning side

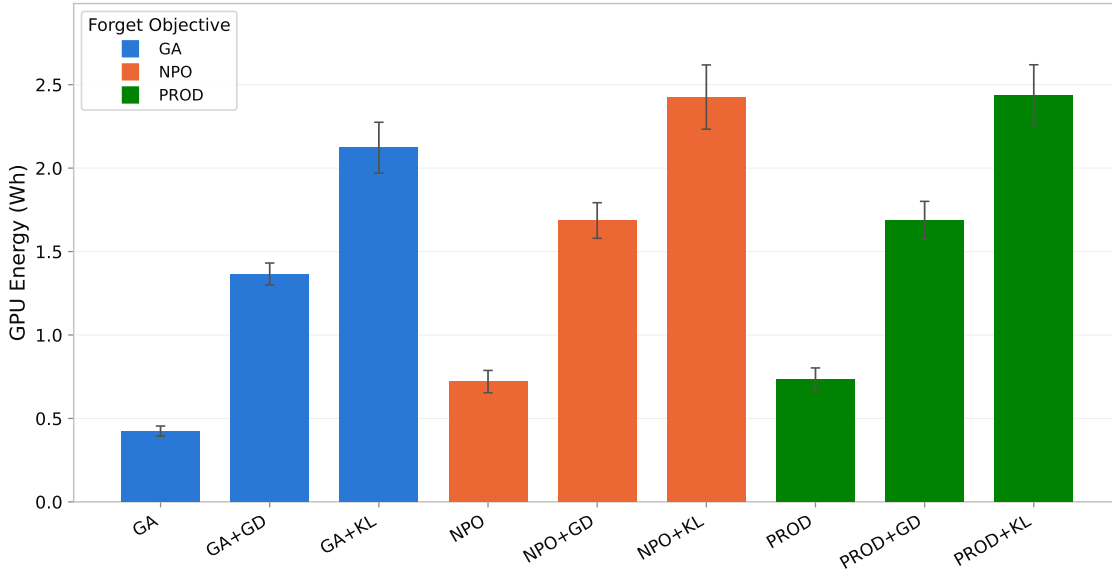


Figure 5.4: Measured GPU energy for secret unlearning runs. Bars show the mean across Llama and Qwen; error bars span the two model-specific values.

effects spill over into neighboring code, we use ForgetEval, which runs functional correctness tests on the functions and classes in which the secrets were injected. We specifically examine whether the location of the injected secret affects these failures. Figure 5.5 shows that functions and classes whose secret is planted in the documentation fail at a substantially lower rate than units whose secret is embedded in the implementation body. Averaged across the 18 model–technique combinations at pass@10, the failure rate was $12.1\% \pm 13.3$ percentage points for code-located secrets, compared with $2.8\% \pm 3.6$ percentage points for documentation-located secrets. At pass@1, the gap was even wider: $46.5\% \pm 18.8$ versus $14.1\% \pm 10.6$, respectively. The reported standard deviations are sample standard deviations across the 18 model–technique combinations.

We verify this association separately for each combination using a two-sided Fisher exact test. Because 18 model–technique combinations were tested at each pass@ k , Benjamini–Hochberg correction was applied within each pass@ k to control the false discovery rate. After correction, secret location was significantly associated with ForgetEval failure in 15 of 18 combinations at pass@1, 8 of 18 at pass@5, and 4 of 18 at pass@10. The declining number of significant associations at larger k is expected because additional generations provide more opportunities to produce a passing solution. Consequently, fewer tasks remain failed at pass@5 and pass@10. Therefore, the decline does not necessarily indicate that the secret location becomes irrelevant; rather, persistent failures become less frequent and harder to distinguish statistically.

Takeaway: Utility preservation around the unlearning target depends on the location. Unlearning secrets from documentation affects utility less than unlearning secrets from the implementation body, particularly in the single-generation setting (ForgetEval pass@1).

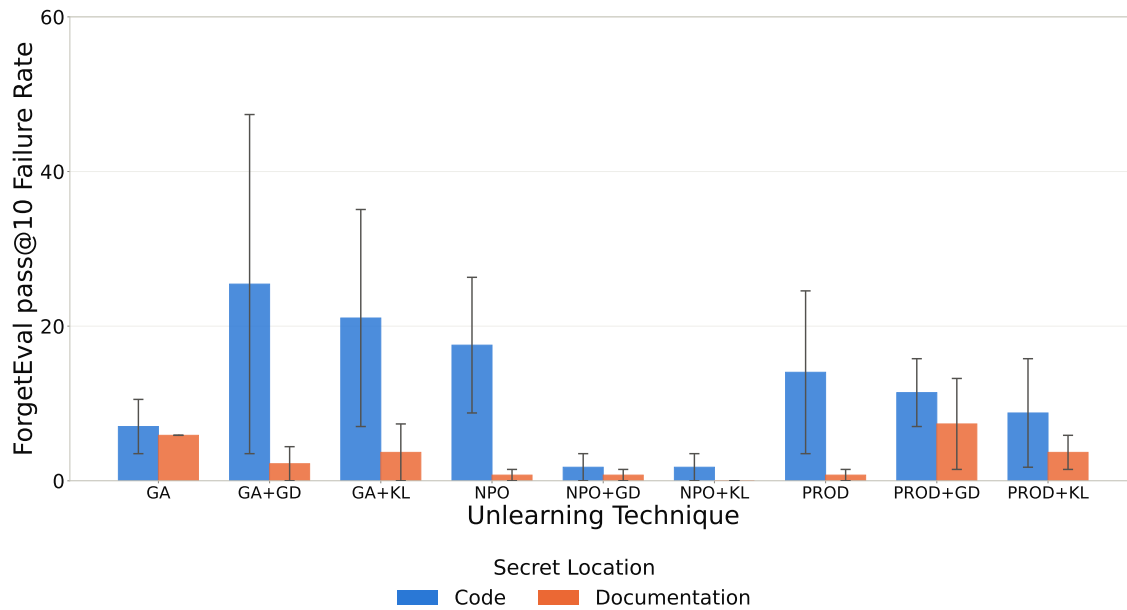


Figure 5.5: ForgetEval pass@10 failure rates by secret location for the selected secret setting checkpoints. Bars report the mean failure rate across Llama and Qwen, while error bars span the two model-specific rates. For each model–technique pair, the checkpoint maximizing the equally weighted mean of UtilityEval pass@10 and Forget-Quality@10 was selected.

Main Finding: The most favorable utility–forgetting trade-off varies across models and training duration. Plain GA and PROD can provide strong trade-offs with lower energy consumption, whereas NPO is generally safer across epochs when combined with retain objectives, particularly KL. Retain objective variants of GA and PROD show reduced stability and incur higher energy costs than their forget-only counterparts.

5.2 RQ2: Transferability of Machine Unlearning to Code Units

While the first research question investigates the trade-offs of MU techniques for secrets injected into code, this research question focuses on structured code units. Specifically, it examines whether the forget quality and utility preservation observed for secrets transfer to these units. Instead of isolating and unlearning secrets within functions and classes, we unlearn entire code segments, simulating the removal of copyrighted or malicious code. The results differ fundamentally once entire code units are subjected to unlearning, which suggests that these techniques may have limited effectiveness beyond short token sequences. As Table 5.2 shows, GA exhibits the same general trend observed in secret unlearning: stronger forgetting comes at the cost of utility. However, the utility degradation is substantially more severe for code units. By epoch 3, the GD and KL variants retain at most 7.1% utility, making the models unusable. Base GA preserves utility best on both models by retaining at least 64.6% at epoch 3, yet its forget quality peaks at only 53.6%, far below the results obtained in the secret unlearning setup. Similar to secret unlearning,

Table 5.2: Code unit unlearning after three epochs. All values are percentages. Green cells meet the 80% target, while red cells indicate near-zero performance ($\leq 10\%$). FQ denotes Forget-Quality@10, and Utility denotes UtilityEval pass@10. Higher values indicate better performance for both metrics.

Model	Metric	GA			NPO			PROD		
		Base	+GD	+KL	Base	+GD	+KL	Base	+GD	+KL
Llama	FQ	28.3	99.8	99.5	23.4	8.2	5.3	84.0	87.5	87.7
	Utility	83.8	0.0	0.0	69.7	92.9	96.0	0.0	0.0	0.0
Qwen	FQ	53.6	74.8	100.0	17.1	4.4	9.4	96.6	97.9	98.5
	Utility	64.6	7.1	0.0	62.6	97.0	99.0	0.0	0.0	0.0

NPO yields the most stable behavior: its retain objective variants maintain utility above 92.9% at every epoch, and even the base NPO never falls below 62.6%. This stability, however, comes at the cost of minimal forgetting, with forget quality never exceeding 23.4% across the NPO family. PROD exhibits the most severe degradation, reducing utility to exactly zero at every epoch on both models. We attribute this to its probability-distribution-based objective, which penalizes necessary syntactic structure. As a result, the generated code may be unexecutable, as illustrated in Listing 5.2. As shown in Figure 5.6, there is a clear gap between code unit and secret unlearning. Secret unlearning results cluster in the top-right corner, which indicates a favorable trade-off between forget quality and utility retention. In contrast, code unit configurations that achieve comparable forget quality lose almost all utility, while those that preserve utility at a level similar to the secret setup achieve substantially lower forget quality. To ensure that differences in evaluation metrics do not drive the observed gap, we recompute all similarities with normalized Levenshtein distance instead of chrF and BLEU and observe the same trends (Appendix Figure 2).

```

1 def synthetic_eval_function_aiF2M9(
2     ...
3     <|file_sep|><|fim_prefix|>/src/retain_function_GGz1x5.py<|fim_suffix|>
4     lif password_length >= len(set(characters)):
5         password_strength = "strong"
6     <|file_sep|>/src/random_integers.py
7     """Compose simple graphs from nodes and edges."""

```

Listing 5.2: PROD-unlearned generation from Qwen at epoch 1 (UtilityEval, code-unit setting; highlighted segment). The model generates a file-separator token, hallucinates an unrelated file path, resulting in an unexecutable solution. The task prompt is omitted for clarity.

Main Finding: The favorable unlearning–utility trade-offs observed for secrets do not transfer to entire code units. PROD eliminates utility, GA substantially degrades utility once effective forgetting is achieved, and NPO preserves utility only at low forget quality. Thus, none of the evaluated techniques provides an effective trade-off for code unit unlearning.

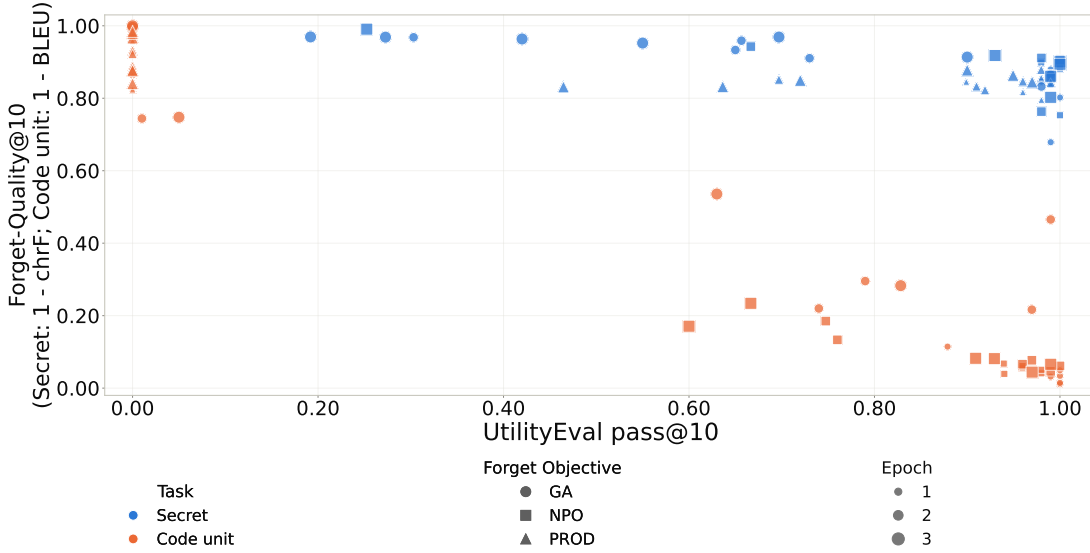


Figure 5.6: Utility–forgetting trade-offs for all secret and code unit checkpoints across both models. For clarity, the visualization of the retain objective is omitted. Higher values on both axes are preferred.

5.3 RQ3: Effects of Retain Set Size and Forget–Retain Objective Ordering

After observing the practical applicability of unlearning secrets embedded in code segments, this research question investigates whether the trade-off between forget quality and utility retention can be further optimized. To this end, we analyze the effects of varying the retain set size and changing the ordering of the forget and retain objectives. Previous experiments randomly order retain and forget batches and use a retain set equal in size to the forget set. To answer this research question, we instead order batches by objective, either performing forgetting before retaining or retaining before forgetting. We also double the retain set size and compare the effects of these modifications against the default secret unlearning setup. The corresponding averaged results across models and epochs are reported in Appendix Table 4.

Forget First. Ordering the unlearning batches as forget followed by retain affects the utility–forgetting trade-off differently across forget objectives. Relative to the base secret setup with random ordering, Forget First substantially reduces the late-epoch utility collapse of the GA variants, while causing a modest reduction in forget quality. The NPO variants largely preserve their utility advantage and obtain small improvements in forget quality. By contrast, the response of PROD is less consistent and can involve substantial utility degradation without a corresponding improvement in forgetting.

As shown in Figure 5.7, the largest utility improvement occurs in the GA family. Forget First improves UtilityEval score by 22.30 ± 19.20 percentage points for GA+GD and 28.36 ± 24.85 percentage points for GA+KL. These gains are concentrated at later epochs: the improvement is only 0.0–1.0 percentage points at epoch 1, but increases to 32.0–52.5 percentage points at epoch 3 across the two models. As illustrated in Figure 5.8, Forget First moves the GA variants toward

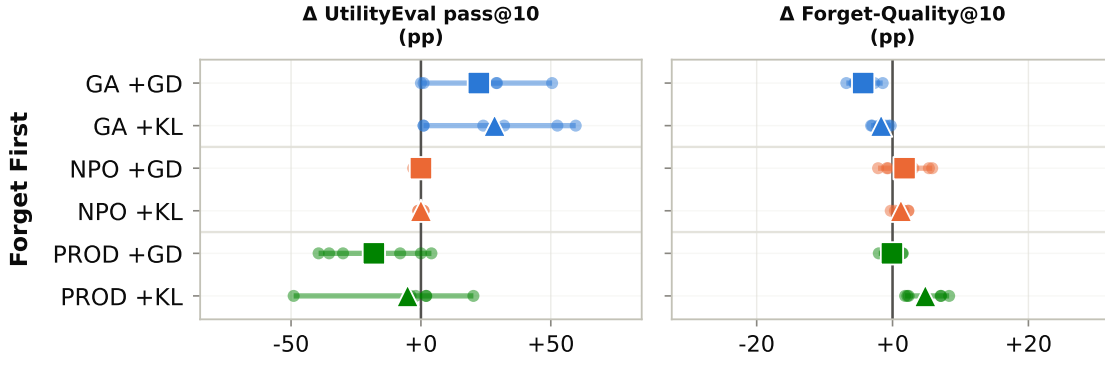


Figure 5.7: Effects of Forget First relative to random ordering in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers their means, and horizontal lines the observed ranges.

the high-utility region. This utility improvement is accompanied by a slight reduction in forget quality of 4.32 ± 1.96 percentage points for GA+GD and 1.68 ± 1.23 percentage points for GA+KL. The NPO family is considerably less affected by the change in ordering. The mean UtilityEval differences are effectively zero for both NPO+GD and NPO+KL, while their forget quality increases by 1.77 ± 3.43 and 1.23 ± 1.06 percentage points, respectively. In contrast, Forget First yields an unfavorable, less predictable trade-off for PROD. For PROD+GD, UtilityEval decreases by 18.12 ± 19.04 percentage points, while the forget quality change is negligible. PROD+KL improves forget quality by 4.83 ± 2.99 percentage points, but its utility effect remains highly variable and averages -5.13 ± 23.15 percentage points.

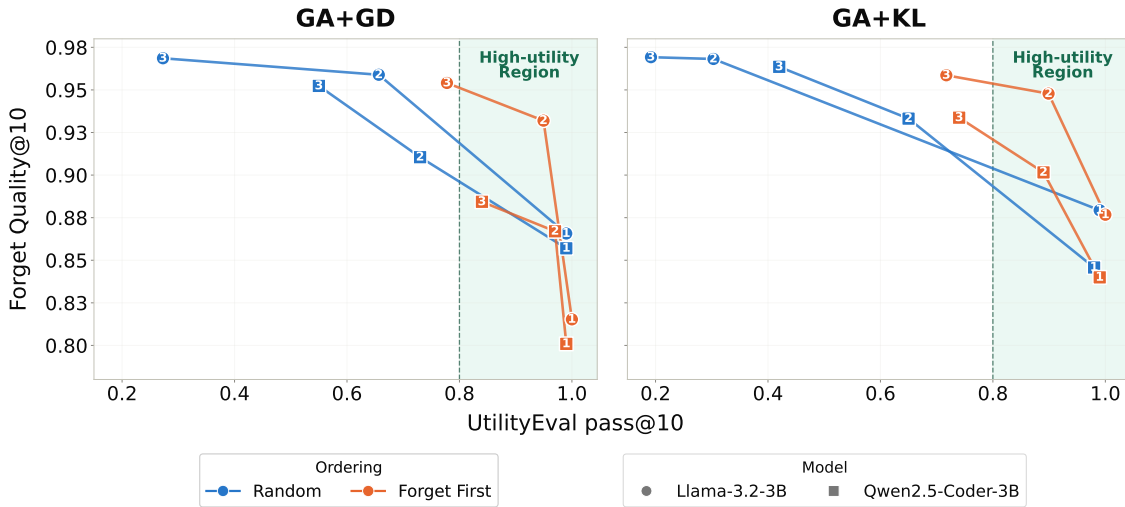


Figure 5.8: Effect of Forget First ordering on the GA-family utility-forgetting trade-off. Numbers identify epochs, and lines connect epochs 1–3 for the same model and ordering. The shaded region denotes UtilityEval of at least 0.80.

Takeaway: Forget First is most beneficial for limiting the utility collapse of GA, has comparatively little effect on the already stable NPO family, and does not provide a reliable improvement for PROD.

Retain First. Retain First produces outcomes similar to results obtained under the Forget First setting. GA-based approaches again show a stabilizing effect on utility, with slightly larger utility gains and more noticeable reductions in forget quality. NPO is the least affected forget objective, with only modest utility losses alongside slight improvements in forget quality. In contrast, PROD consistently performs worse under objective ordering.

As shown in Figure 5.9, Retain First improves UtilityEval for GA+GD and GA+KL by 24.81 ± 24.47 and 33.37 ± 27.45 percentage points, respectively. However, these gains are accompanied by average reductions in forget quality of 12.28 ± 7.25 percentage points for GA+GD and 7.89 ± 6.70 percentage points for GA+KL. NPO largely retains its stability: NPO+GD and NPO+KL lose 6.07 ± 8.68 and 1.68 ± 1.64 percentage points of utility, while improving forget quality by 2.98 ± 4.75 and 1.74 ± 3.68 percentage points. PROD shows a less favorable trade-off, particularly for PROD+GD, for which utility decreases by 61.32 ± 13.87 percentage points on average. Both PROD variants also exhibit slight reductions in forget quality of approximately 0.5–0.6 percentage points.

Takeaway: Retain First strengthens the utility stabilizing effect for GA, but at a clear cost to forget quality. NPO is the least affected forget objective and largely preserves its stable behavior, whereas PROD, particularly PROD+GD, can experience utility collapse to unusable levels.

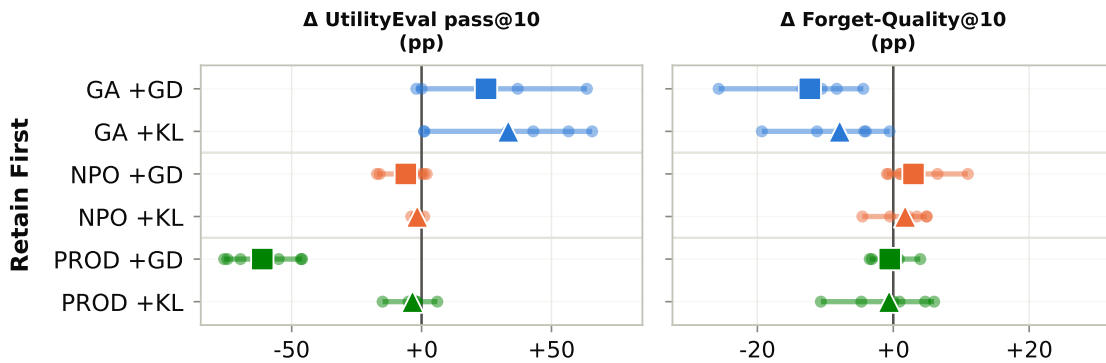


Figure 5.9: Effects of Retain First relative to random ordering in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers their means, and horizontal lines the observed ranges.

Double Retain Set. Finally, we analyze how increasing the retain set size interacts with different MU techniques. We observe that a larger retain set does not consistently preserve more utility. Only the GA family benefits from doubling the retain set, and the resulting utility gains are limited. In contrast, NPO and PROD consistently suffer from reduced utility, particularly their GD

variants. Increasing the retain set size has only a modest effect on forget quality, while it substantially increases GPU energy consumption and training time, thus providing limited practical benefit.

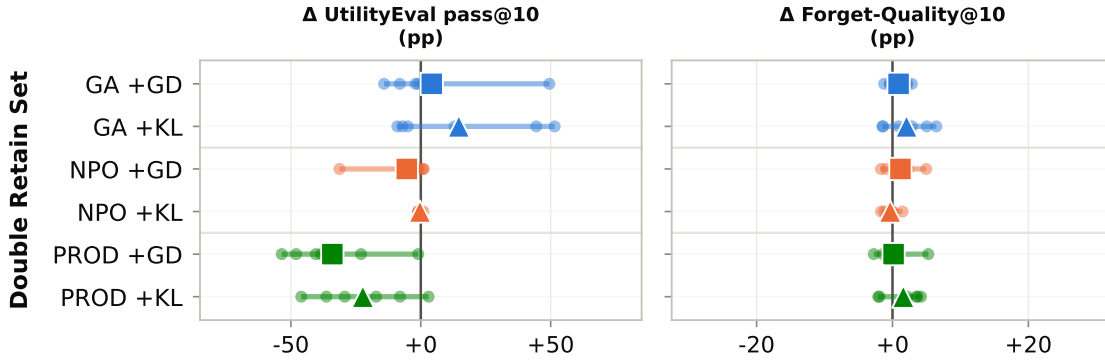


Figure 5.10: Effects of doubling retain set size in the secret setting. Differences are matched by model, objective family, and epoch. Small points show the six observations (two models $\times$ three epochs), large markers show their means, and horizontal lines show the observed ranges.

Figure 5.10 shows that, as in the ordering setups, GA+GD and GA+KL improve utility, although to a lesser extent, by 4.21 ± 22.87 and 14.65 ± 27.08 percentage points, respectively. These improvements are accompanied by only minor gains in forget quality of 0.90 ± 1.57 and 2.06 ± 3.31 percentage points. By contrast, both NPO and PROD suffer from reduced utility, most notably PROD+GD, whose utility decreases by 34.05 ± 19.22 percentage points on average. These families likewise show no substantial changes in forget quality, with mean differences ranging from -0.34 to $+1.59$ percentage points. Compared with the ordering setups, doubling the retain set causes substantially higher computational costs. As Figure 5.11 shows, it increases resource consumption across all techniques. Averaged across techniques, GPU energy consumption increases by 79.9%, while training time increases by 61.6%.

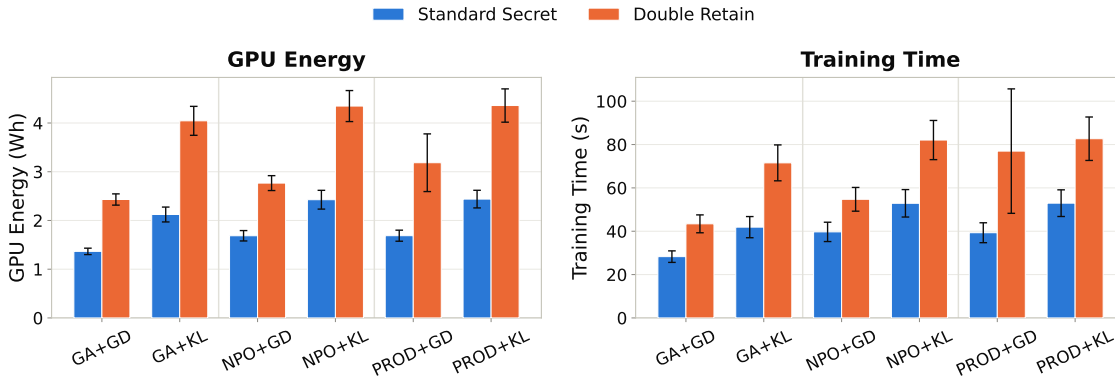


Figure 5.11: GPU energy consumption and training time of standard secret and double retain set unlearning setups. Bars report the mean over Llama and Qwen, while error bars span the two model measurements.

Takeaway: Doubling the retain set modestly improves utility only for GA and reduces utility for NPO and PROD, with little effect on forget quality. These limited gains come at the cost of substantially higher computational requirements.

Main Finding: Changing the forget and retain ordering affects the utility–forgetting trade-off in a technique-dependent manner. Objective ordering can substantially stabilize GA, has limited effects on the already stable NPO, and generally harms PROD. Overall, Forget First emerges as the more favorable ordering technique. In contrast, doubling the retain set mostly degrades performance, provides only limited benefits for GA, and substantially increases energy consumption and training time.

5.4 Ablation Studies

Comparison of Full-Parameter and LoRA Unlearning

All preceding unlearning experiments in this study update only LoRA adapters. We therefore conduct a focused ablation to examine whether expanding the parameter-update scope changes the unlearning–utility trade-off. Specifically, we compare LoRA-based and full-parameter GA unlearning on Qwen using the MOCHI benchmark. In the LoRA setting, unlearning is applied only to the adapters obtained during fine-tuning. In contrast, in the full-parameter setting, unlearning is applied to the base model merged with the fine-tuned adapters. We tune the full-parameter unlearning configuration separately since LoRA and full-parameter optimization may require different learning rates [76, 77]. The search covers learning rates from 5×10^{-7} to 5×10^{-5} . Selected trajectories are shown in Appendix Figure 1. Based on this search, we use 2×10^{-5} for the full-parameter comparison.

Table 5.3: Full-parameter GA relative to LoRA-based GA on Qwen.

Epoch	Δ Forget-Quality@10 [95% CI], pp	Δ UtilityEval pass@10 [95% CI], pp	Δ GPU Energy (%)	Δ Time (%)
1	+7.62 [+4.53, +11.00]	+1.00 [0.00, +3.00]	–	–
2	+0.13 [–2.16, +2.40]	+1.00 [–2.00, +5.00]	–	–
3	–2.78 [–4.18, –1.37]	–1.00 [–7.00, +5.00]	–	–
<i>Mean / full run</i>	+1.66 [+0.28, +3.06]	+0.33 [–2.67, +3.67]	+59.9	+73.9

Differences are full-parameter GA minus LoRA-based GA. GPU energy and time cover the complete three-epoch runs. Performance differences report paired 95% bootstrap CIs from 10,000 resamples.

Our results show that full-parameter GA does not provide a meaningful improvement in forget quality or utility retention, while substantially increasing training time and GPU energy consumption. As illustrated in Table 5.3, full-parameter GA provides a minor early advantage after the first epoch by improving forget quality by 7.62 pp (95% CI: [4.53, 11.00]) relative to LoRA. This advantage decreases to 0.13 pp after the second epoch. By the third epoch, the direction reverses and forget quality reduces by 2.78 pp (95% CI: [–4.18, –1.37]) compared to LoRA. Averaged across epochs, full-parameter GA retains a slight forget quality advantage of 1.66 pp (95% CI: [0.28, 3.06]), although this advantage is not consistent throughout unlearning. In contrast,

the UtilityEval differences range from -1.00 to $+1.00$ pp and remain relatively small. The results provide no clear evidence that either parameterization better preserves fine-tuning utility. Full-parameter unlearning nevertheless requires substantially greater computational resources, consistent with the established efficiency advantages of LoRA [31, 78, 79]. It increases measured GPU energy by 59.9% and training duration by 73.9%. Considering the increased energy consumption, the modest improvement in forget quality, and the lack of clear improvement in utility retention, full-parameter GA is less attractive for MU.

Limitations and Future Work

Internal validity. A first threat to internal validity concerns the dataset curation stage, particularly the similarity between code units. In this stage, another important consideration is the extent to which the resulting code represents realistic cases while minimizing overlap with code knowledge already present in the models. To mitigate this threat, we apply threshold-based filtering across multiple dimensions and conduct an initial memorization analysis of the curated samples. A second threat concerns the experimental configuration. Throughout the study, we use hyperparameters validated in prior work on MU for code [23]; however, these values may not be equally suitable for our forget and retain sets. We mitigate this risk through small-scale grid searches. Fine-tuning and unlearning may also introduce variance. Since we do not run multiple seeds, observed differences between methods or ordering strategies may partly reflect randomness. The evaluation setup introduces additional threats. Although the synthetically generated test suites increase diversity by covering different inputs and edge cases, this diversity may not accurately represent the distribution of real-world programs and developer-written tests. We try to mitigate this by adding a mutation-hardening step that uses mutation scores as a guide. Moreover, CodeCarbon follows the general pattern of energy use, but consistently underestimates consumption by about 20–30%, which suggests that CodeCarbon is useful for relative comparisons, but less reliable for exact energy consumption [80]. This is the main reason why we interpret energy results as general trends rather than definitive truth. Finally, one of the most important internal threats is the choice of metrics. Since there is no established metric for the unlearning field, we decided to use the text similarity metrics BLEU and chrF, which may not effectively quantify the unlearning effect.

External validity. The most significant external threat to validity is the use of a synthetic dataset rather than code units collected from open-source repositories. However, the generated code is expected to exhibit statistical characteristics similar to human-written code because the LLMs used for generation were trained on large-scale corpora of publicly available source code. Therefore, the synthetic code may follow a distribution comparable to real-world code, although this similarity cannot be guaranteed. Another limitation is the selection of models. Due to resource limitations, we evaluate only two small LLMs with 3B parameters. To address this generalizability concern, we evaluate one general-purpose LLM and one code-specific LLM. In addition, this study only considers Python, and its generalizability to other programming languages is therefore uncertain. Since we focus only on functions and classes, it is also unclear whether the findings hold for arbitrary code fragments without a clear structure. A final limitation concerns method coverage: we do not evaluate every existing MU technique. However, our selection includes most techniques previously applied to code and extends them to new combinations of objectives.

As future work, several directions could help address the limitations identified above. To assess the generalizability of our findings more comprehensively, future studies should evaluate a broader range of LLMs from different model families and across a wider spectrum of model sizes.

Repeating experiments with multiple random seeds would further help quantify the variance introduced by fine-tuning and unlearning and distinguish systematic effects from randomness. Extending the evaluation to additional programming languages would also enable investigation of whether language-specific characteristics influence unlearning effectiveness and allow systematic comparisons across languages. Another promising direction is to examine unlearning approaches that do not require an additional training step, particularly from the perspective of the trade-off between computational cost, forget quality, and retained utility. Existing MU techniques primarily target specific, isolated instances. Future work could extend these techniques to broader concepts or related groups of code units. The MOCHI benchmark could be extended in future work to cover more realistic unlearning scenarios, such as the removal of deprecated code, outdated APIs, or entire repositories. Finally, new unlearning algorithms should be developed or adapted specifically for removing complete code units while explicitly accounting for the structural and semantic properties of code. Nevertheless, not all limitations identified in this study can be addressed, as some stem from broader open challenges in MU for code. Among these are the lack of established evaluation metrics and the difficulty of defining and verifying complete forgetting.

Conclusion

The main goal of this thesis is to characterize the trade-off between forget quality, coding capability retention, and energy consumption when applying MU for code generation tasks. We identify four factors that jointly shape this trade-off: the choice of MU technique, the type of content targeted for removal, the size of the retain set, and the ordering of the forget and retain objectives. To analyze these, we introduce MOCHI, a benchmark for MU of code with hidden information, which includes 600 synthetic functions and classes with mutation-hardened test suites.

We conduct two separate unlearning experiments on MOCHI. First, we target secrets injected into functions and classes, then compare this against unlearning the corresponding code units as a whole. Our results indicate a significant gap between the secret and code unit unlearning settings. Existing MU methods work as intended for removing secrets: NPO variants, particularly NPO+KL, provide the most stable trade-off in most scenarios by balancing forget quality and utility retention. MU methods without a retain objective also offer a favorable trade-off, with lower energy consumption than techniques that utilize both a forget and retain objective simultaneously. However, their behavior is less consistent across models and can change considerably over the course of epochs, especially for GA. In contrast, code unit unlearning configurations fail to achieve comparable success. When forget quality equivalent to the secret setting is reached, utility degrades to an unusable level, and preserving utility makes the unlearning process ineffective. This suggests that future work should focus on new algorithms that achieve unlearning without compromising the coding capabilities of the models. The retain set size and objective ordering (Forget First, Retain First, and Random) experiments address a largely unexplored dimension in the MU literature. Our results suggest that the forget objective remains the dominant factor in determining the outcome. GA-based approaches become more stable with a larger retain set and ordered objectives, though this generally comes at the cost of some forget quality. For NPO, these changes have only a limited effect on the outcome. Placing forget set optimization first, with KL as the retention signal, is nevertheless slightly beneficial for NPO and shows a more noticeable benefit for GA, whereas PROD responds negatively to these modifications and typically loses both forget quality and utility retention.

Overall, our findings demonstrate that the effectiveness of MU for code depends not only on the unlearning technique itself, but also on what is being forgotten and how the unlearning procedure is configured. While existing methods can effectively remove localized information such as secrets, unlearning complete code units while preserving model utility remains an open challenge. By systematically evaluating content type, retain set size, objective ordering, and energy consumption across two removal scenarios, this thesis provides a basis for selecting and configuring MU methods for code generation and identifies the limitations that future approaches need to address. The replication package for this work can be found at <https://github.com/DogukanBaysal/Code-Unlearning>

Bibliography

- [1] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, pp. 1 – 72, 2024.
- [2] S. B. Hossain, N. Jiang, Q. Zhou, X. Li, W.-H. Chiang, Y. Lyu, H. Nguyen, and O. Tripp, "A deep dive into large language models for automated bug localization and repair," *Proceedings of the ACM on Software Engineering*, vol. 1, pp. 1471 – 1493, 2024.
- [3] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," *IEEE Transactions on Software Engineering*, vol. 50, pp. 85–105, 2023.
- [4] D. Zhang, P. Finckenberg-Broman, T. Hoang, S. Pan, Z. Xing, M. Staples, and X. Xu, "Right to be forgotten in the era of large language models: Implications, challenges, and solutions," 2024.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, and N. J. et al., "Evaluating large language models trained on code," 2021.
- [6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020.
- [7] C. Zhang, H. Zhong, K. Zhang, C. Chai, R. Wang, X. Zhuang, T. Bai, Q. Jiantao, L. Cao, J. Fan, Y. Yuan, G. Wang, and C. He, "Harnessing diversity for important data selection in pretraining large language models," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [8] C. Zhang, C. Yang, Q. Tan, J. Liu, A. Li, Y. Wang, J. Lu, J. Wang, and G. Yuan, "Towards memory-efficient and sustainable machine unlearning on edge using zeroth-order optimizer," in *Proceedings of the Great Lakes Symposium on VLSI 2025*, GLSVLSI '25, (New York, NY, USA), p. 227–232, Association for Computing Machinery, 2025.
- [9] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, T. Liu, M. Tian, D. Kocetkov, A. Zucker, Y. Belkada, Z. Wang, Q. Liu, D. Abulkhanov, I. Paul, Z. Li, W.-D. Li, M. Risdal, J. Li, J. Zhu, T. Y. Zhuo, E. Zheltonozhskii, N. O. O. Dade, W. Yu, L. Krauß, N. Jain, Y. Su, X. He, M. Dey, E. Abati, Y. Chai, N. Muennighoff, X. Tang, M. Oblokulov, C. Akiki, M. Marone, C. Mou, M. Mishra, A. Gu, B. Hui, T. Dao, A. Zebaze,

- O. Dehaene, N. Patry, C. Xu, J. McAuley, H. Hu, T. Scholak, S. Paquet, J. Robinson, C. J. Anderson, N. Chapados, M. Patwary, N. Tajbakhsh, Y. Jernite, C. M. Ferrandis, L. Zhang, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "Starcode 2 and the stack v2: The next generation," 2024.
- [10] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024.
- [11] A. Al-Kaswan, M. Izadi, and A. van Deursen, "Traces of Memorisation in Large Language Models for Code," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, (New York, NY, USA), Association for Computing Machinery, 2024.
- [12] J. He, M. Vero, G. Krasnopolska, and M. Vechev, "Instruction tuning for secure code generation," in *Proceedings of the 41st International Conference on Machine Learning, ICML'24*, (Vienna, Austria), JMLR.org, 2024.
- [13] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," 2020.
- [14] A. Ginart, M. Y. Guan, G. Valiant, and J. Zou, "Making ai forget you: Data deletion in machine learning," 2019.
- [15] S. Neel, A. Roth, and S. Sharifi-Malvajerdi, "Descent-to-delete: Gradient-based methods for machine unlearning," in *Proceedings of the 32nd International Conference on Algorithmic Learning Theory*, vol. 132 of *Proceedings of Machine Learning Research*, pp. 931–962, PMLR, 2021.
- [16] European Union, *Charter of Fundamental Rights of the European Union*, vol. 53. Brussels: European Union, 2010.
- [17] GDPR.eu, "Right to be forgotten." <https://gdpr.eu/right-to-be-forgotten/>. Accessed March 2, 2026.
- [18] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 of the european parliament and of the council." Official Journal of the European Union, L 119, 2016. General Data Protection Regulation, Articles 12(3) and 17.
- [19] S. Liu, Y. Yao, J. Jia, S. Casper, N. Baracaldo, P. Hase, Y. Yao, C. Y. Liu, X. Xu, H. Li, K. R. Varshney, M. Bansal, S. Koyejo, and Y. Liu, "Rethinking machine unlearning for large language models," 2024.
- [20] M. V. Waerebeke, M. Lorenzi, G. Neglia, and K. Scaman, "When to forget? complexity trade-offs in machine unlearning," 2025.
- [21] P. Maini, Z. Feng, A. Schwarzschild, Z. C. Lipton, and J. Z. Kolter, "Tofu: A task of fictitious unlearning for llms," 2024.
- [22] R. Geng, M. Geng, S. Wang, H. Wang, Z. Lin, and D. Dong, "Mitigating sensitive information leakage in llms4code through machine unlearning," 2025.
- [23] X. Jiang, Y. Dong, Z. Fang, Y. Ma, T. Wang, R. Cao, B. Li, Z. Jin, W. Jiao, Y. Li, and G. Li, "Large language model unlearning for source code," 2025.
- [24] Z. Chu, Y. Wan, Z. Zhang, D. Wang, Z. Yang, H. Zhang, P. Zhou, X. Shi, H. Jin, and D. Lo, "Scrub it out! erasing sensitive memorization in code language models via machine unlearning," *arXiv preprint arXiv:2509.13755*, 2025.

- [25] N. Li, C. Zhou, Y. Gao, H. Chen, A. Fu, Z. Zhang, and Y. Shui, "Machine unlearning: Taxonomy, metrics, applications, challenges, and prospects," 2024.
- [26] Z. Liu, G. Dou, Z. Tan, Y. Tian, and M. Jiang, "Machine unlearning in generative ai: A survey," 2024.
- [27] A. Warnecke, L. Pirch, C. Wressnegger, and K. Rieck, "Machine unlearning of features and labels," 2023.
- [28] J. Yao, E. Chien, M. Du, X. Niu, T. Wang, Z. Cheng, and X. Yue, "Machine unlearning of pre-trained large language models," 2024.
- [29] S. Cha, S. Cho, D. Hwang, and M. Lee, "Towards robust and parameter-efficient knowledge unlearning for llms," 2025.
- [30] Y. Kim, E. Kim, B. Chang, and J. Choe, "Improving fisher information estimation and efficiency for loRA-based LLM unlearning," in *Second Conference on Language Modeling*, 2025.
- [31] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021.
- [32] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, "Parameter-efficient fine-tuning for large models: A comprehensive survey," 2024.
- [33] Y. Yao, X. Xu, and Y. Liu, "Large language model unlearning," in *Advances in Neural Information Processing Systems* (A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, eds.), vol. 37, pp. 105425–105475, Curran Associates, Inc., 2024.
- [34] R. Zhang, L. Lin, Y. Bai, and S. Mei, "Negative preference optimization: From catastrophic collapse to effective unlearning," 2024.
- [35] B. Liu, Q. Liu, and P. Stone, "Continual learning and private unlearning," 2022.
- [36] J. Cui, Z. Tian, Z. Zhong, X. Qi, B. Yu, and H. Zhang, "Decoupled kullback-leibler divergence loss," *ArXiv*, vol. abs/2305.13948, 2023.
- [37] Y. Liang, Q. Li, S. Wang, and W. Cao, "When forgetting builds reliability: Llm unlearning for reliable hardware code generation," 2025.
- [38] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, and S. A. et al., "Gpt-4 technical report," 2024.
- [39] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (P. Isabelle, E. Charniak, and D. Lin, eds.), (Philadelphia, Pennsylvania, USA), pp. 311–318, Association for Computational Linguistics, July 2002.
- [40] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in *Proceedings of the Tenth Workshop on Statistical Machine Translation* (O. Bojar, R. Chatterjee, C. Federmann, B. Haddow, C. Hokamp, M. Huck, V. Logacheva, and P. Pecina, eds.), (Lisbon, Portugal), pp. 392–395, Association for Computational Linguistics, Sept. 2015.
- [41] W. Shi, A. Ajith, M. Xia, Y. Huang, D. Liu, T. Blevins, D. Chen, and L. Zettlemoyer, "Detecting pretraining data from large language models," 2024.
- [42] K. Tirumala, A. H. Markosyan, L. Zettlemoyer, and A. Aghajanyan, "Memorization without overfitting: Analyzing the training dynamics of large language models," 2022.

- [43] J. Jang, D. Yoon, S. Yang, S. Cha, M. Lee, L. Logeswaran, and M. Seo, "Knowledge unlearning for mitigating privacy risks in language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (A. Rogers, J. Boyd-Graber, and N. Okazaki, eds.), (Toronto, Canada), pp. 14389–14408, Association for Computational Linguistics, July 2023.
- [44] Z. Cai, Y. Tan, and M. S. Asif, "Targeted unlearning with single layer unlearning gradient," in *Proceedings of the 42nd International Conference on Machine Learning* (A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff, and J. Zhu, eds.), vol. 267 of *Proceedings of Machine Learning Research*, pp. 6257–6290, PMLR, 13–19 Jul 2025.
- [45] J. Foster, S. Schoepf, and A. Brintrup, "Fast machine unlearning without retraining through selective synaptic dampening," 2023.
- [46] J. Chen and D. Yang, "Unlearn what you want to forget: Efficient unlearning for llms," 2023.
- [47] B. Courty, V. Schmidt, S. Luccioni, Goyal-Kamal, MarionCoutarel, B. Feld, J. Lecourt, Liam-Connell, A. Saboni, Inimaz, supatomic, M. Léval, L. Blanche, A. Cruveiller, ouminasara, F. Zhao, A. Joshi, A. Bogroff, H. de Lavoreille, N. Laskaris, E. Abati, D. Blank, Z. Wang, A. Catovic, M. Alencon, M. Stęchły, C. Bauer, L. O. N. de Araújo, JPW, and MinervaBooks, "mlco2/codecarbon: v2.4.1," May 2024.
- [48] H. Husain, H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," *ArXiv*, vol. abs/1909.09436, 2019.
- [49] M. Rahman, S. Khatoonabadi, and E. Shihab, "Openclassgen: A large-scale corpus of real-world python classes for llm research," 2025.
- [50] A. LeClair and C. McMillan, "Recommendations for datasets for source code summarization," *ArXiv*, vol. abs/1904.02660, 2019.
- [51] V. Majdinasab, A. Nikanjam, and F. Khomh, "Trained without my consent: Detecting code inclusion in language models trained on code," *ACM Transactions on Software Engineering and Methodology*, vol. 34, p. 1–46, Apr. 2025.
- [52] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, and A. A. et al., "Openai gpt-5 system card," 2026.
- [53] J. McCabe, Thomas J., "Software quality metrics to identify risk." Presentation to the Department of Homeland Security Software Assurance Working Group, Jan. 2008. Last edited for content in November 2008. Archived from the original on March 29, 2022.
- [54] T. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [55] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," 2021.
- [56] J. George, "Large language models for automated software test generation," *International Journal of Information Technology Research Studies (IJITRS)*, 2026.
- [57] G. Petrović, M. Ivanković, G. Fraser, and R. Just, "Practical mutation testing at scale: A view from google," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3900–3912, 2022.
- [58] G. Grano, F. Palomba, and H. C. Gall, "Lightweight assessment of test-case effectiveness using source-code-quality indicators," *IEEE Transactions on Software Engineering*, vol. 47, no. 4, pp. 758–774, 2021.

- [59] B. H. Smith and L. Williams, "On guiding the augmentation of an automated test suite via mutation analysis," *Empirical Softw. Engg.*, vol. 14, p. 341–369, June 2009.
- [60] H. Chen, A. Waheed, X. Li, Y. Wang, J. Wang, B. Raj, and M. I. Abdin, "On the diversity of synthetic data and its impact on training large language models," 2024.
- [61] Y. Shao, L. Li, Y. Ma, P. Li, D. Song, Q. Cheng, S. Li, X. Li, P. Wang, Q. Guo, H. Yan, X. Qiu, X. Huang, and D. Lin, "Case2code: Scalable synthetic data for code generation," 2025.
- [62] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," 2020.
- [63] N. Godey, É. Clergerie, and B. Sagot, "Anisotropy is inherent to self-attention in transformers," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (Y. Graham and M. Purver, eds.), (St. Julian's, Malta), pp. 35–48, Association for Computational Linguistics, Mar. 2024.
- [64] J. Mu, S. Bhat, and P. Viswanath, "All-but-the-top: Simple and effective postprocessing for word representations," 2018.
- [65] L. Ai, Z. Gong, H. Deshpande, A. Johnson, E. Phung, A. Emami, and J. Hirschberg, "Novascore: A new automated metric for evaluating document level novelty," 2024.
- [66] A. Spiesberger, J. J. Vazquez, N. Pochinkov, T. Gavenčiak, P. Grietzer, G. Leech, and N. Schoots, "Soft contamination means benchmarks test shallow generalization," 2026.
- [67] M. Mishra, M. Stallone, G. Zhang, Y. Shen, A. Prasad, A. M. Soria, M. Merler, P. Selvam, S. Surendran, S. Singh, M. Sethi, X.-H. Dang, P. Li, K.-L. Wu, S. Zawad, A. Coleman, M. White, M. Lewis, R. Pavuluri, Y. Koyfman, B. Lublinsky, M. de Bayser, I. Abdelaziz, K. Basu, M. Agarwal, Y. Zhou, C. Johnson, A. Goyal, H. Patel, Y. Shah, P. Zerfos, H. Ludwig, A. Munawar, M. Crouse, P. Kapanipathi, S. Salaria, B. Calio, S. Wen, S. Seelam, B. Belgodere, C. Fonseca, A. Singhee, N. Desai, D. D. Cox, R. Puri, and R. Panda, "Granite code models: A family of open foundation models for code intelligence," 2024.
- [68] H. Sajnani, V. Saini, J. Svajlenko, C. K. Roy, and C. V. Lopes, "Sourcerercc: Scaling code clone detection to big-code," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, pp. 1157–1168, 2016.
- [69] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.
- [70] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, and A. V. et al., "The llama 3 herd of models," 2024.
- [71] N. Carlini, D. Ippolito, M. Jagielski, K. Lee, F. Tramèr, and C. Zhang, "Quantifying memorization across neural language models," 2023.
- [72] D. Aspinall and M. Just, "'give me letters 2, 3 and 6!': Partial password implementations and attacks," in *Financial Cryptography and Data Security* (A.-R. Sadeghi, ed.), (Berlin, Heidelberg), pp. 126–143, Springer Berlin Heidelberg, 2013.
- [73] U.S. Copyright Office, "What does copyright protect?." <https://www.copyright.gov/help/faq/faq-protect.html>, n.d. Frequently Asked Questions. Accessed: 2026-06-23.

- [74] J. R. Autry, "Toward a definition of striking similarity in infringement actions for copyrighted musical works," *Journal of Intellectual Property Law*, vol. 10, no. 1, p. 113, 2002.
- [75] M. Post, "A call for clarity in reporting BLEU scores," in *Proceedings of the Third Conference on Machine Translation: Research Papers* (O. Bojar, R. Chatterjee, C. Federmann, M. Fishel, Y. Graham, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, C. Monz, M. Negri, A. N  v  ol, M. Neves, M. Post, L. Specia, M. Turchi, and K. Verspoor, eds.), (Brussels, Belgium), pp. 186–191, Association for Computational Linguistics, Oct. 2018.
- [76] R. Shuttleworth, J. Andreas, A. Torralba, and P. Sharma, "Lora vs full fine-tuning: An illusion of equivalence," 2025.
- [77] N. Chen, S. Villar, and S. Hayou, "Learning rate scaling across lora ranks and transfer to full finetuning," 2026.
- [78] A. J. Shreshthi, S. Span  , L. D. Nunzio, R. L. Cesa, C. V. M. Re, and G. C. Cardarilli, "Parameter-efficient fine-tuning of llms for low-power sentiment analysis applications," *JOIV : International Journal on Informatics Visualization*, 2026.
- [79] X. Sun, Y. Ji, B. Ma, and X. Li, "A comparative study between full-parameter and lora-based fine-tuning on chinese instruction data for instruction following large language model," *ArXiv*, vol. abs/2304.08109, 2023.
- [80] R. Fischer, "Ground-truthing ai energy consumption: Validating codecarbon against external measurements," 2025.

Appendix

Table 1: Sensitivity of the Secret pass@10 best-epoch selection to a common hardened UtilityEval and Forget Quality threshold τ . Epoch agreement is calculated among configurations remaining eligible at τ , relative to the 0.80/0.80 selection. Frontier retention is relative to its four Pareto-optimal points.

τ	Qualifying checkpoints	Eligible configurations	Same selected epoch	Frontier retained
0.65	46/54	18/18	17/18	4/4
0.70	40/54	18/18	17/18	4/4
0.75	38/54	18/18	17/18	4/4
0.80	35/54	18/18	18/18	4/4
0.85	22/54	13/18	12/13	4/4
0.90	7/54	5/18	5/5	3/4

Table 2: One-factor-at-a-time sensitivity of the Secret pass@10 selection. One threshold is varied while the other is fixed at 0.80. Epoch agreement is relative to the selected epochs at 0.80/0.80 and is reported among eligible configurations. Frontier retention is relative to the four Pareto-optimal points at 0.80/0.80.

τ	Vary UtilityEval ($\tau_{\text{FQ}} = 0.80$)			Vary Forget Quality ($\tau_{\text{U}} = 0.80$)		
	Eligible	Epoch match	Frontier	Eligible	Epoch match	Frontier
0.65	18/18	18/18	4/4	18/18	17/18	4/4
0.70	18/18	18/18	4/4	18/18	17/18	4/4
0.75	18/18	18/18	4/4	18/18	17/18	4/4
0.80	18/18	18/18	4/4	18/18	18/18	4/4
0.85	18/18	18/18	4/4	13/18	12/13	4/4
0.90	17/18	17/17	4/4	5/18	5/5	3/4

Table 3: Sensitivity of the Secret pass@10 best-epoch selection to baseline-derived filtering at the fixed 0.80 hardened UtilityEval and Forget Quality thresholds. Epoch agreement is relative to the filtered selection. Frontier retention is relative to its four Pareto-optimal points.

Result set	Qualifying checkpoints	Eligible configurations	Same selected epoch	Frontier retained
Filtered	35/54	18/18	18/18	4/4
Unfiltered	36/54	18/18	15/18	3/4

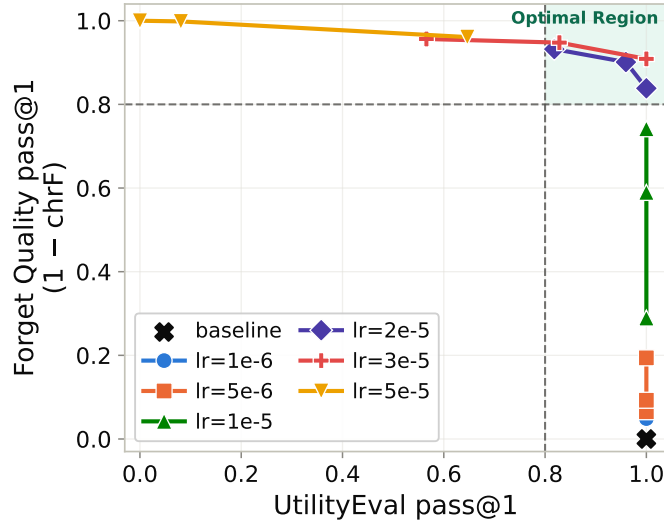


Figure 1: Learning-rate sensitivity of full-parameter GA on Qwen2.5-Coder-3B under the secret unlearning setting. Connected markers represent epochs 1–3.

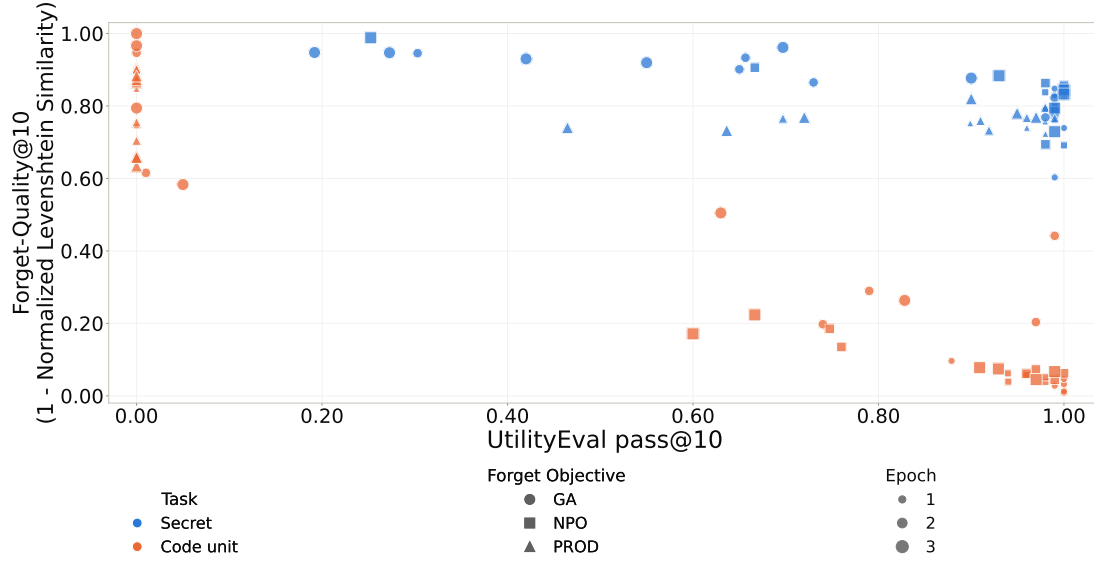


Figure 2: Utility–forgetting trade-offs for all checkpoints across both models, using 1–normalized character-level Levenshtein similarity as a common forget quality measure. For clarity, the visualization of the retain objective is omitted. Higher values on both axes are preferred.

Table 4: Mean effects of alternative secret unlearning settings relative to default random ordering of forget and retain batches. Values report the technique mean $\pm$ sample standard deviation.

Performance differences are averaged across both models and three epochs ($n = 6$ per technique). Positive values indicate improved performance.

Setting	Technique	Δ UtilityEval pass@10 (pp)	Δ Forget Quality@10 (pp)
Forget First	GA+GD	$+22.30 \pm 19.20$	-4.32 ± 1.96
	GA+KL	$+28.36 \pm 24.85$	-1.68 ± 1.23
	NPO+GD	0.00 ± 1.68	$+1.77 \pm 3.43$
	NPO+KL	0.00 ± 0.90	$+1.23 \pm 1.06$
	PROD+GD	-18.12 ± 19.04	-0.09 ± 1.40
	PROD+KL	-5.13 ± 23.15	$+4.83 \pm 2.99$
Retain First	GA+GD	$+24.81 \pm 24.47$	-12.28 ± 7.25
	GA+KL	$+33.37 \pm 27.45$	-7.89 ± 6.70
	NPO+GD	-6.07 ± 8.68	$+2.98 \pm 4.75$
	NPO+KL	-1.68 ± 1.64	$+1.74 \pm 3.68$
	PROD+GD	-61.32 ± 13.87	-0.52 ± 2.76
	PROD+KL	-3.50 ± 6.79	-0.62 ± 6.20
Double Retain Set	GA+GD	$+4.21 \pm 22.87$	$+0.90 \pm 1.57$
	GA+KL	$+14.65 \pm 27.08$	$+2.06 \pm 3.31$
	NPO+GD	-5.39 ± 12.76	$+1.17 \pm 2.42$
	NPO+KL	-0.34 ± 0.82	-0.34 ± 1.11
	PROD+GD	-34.05 ± 19.22	$+0.16 \pm 2.80$
	PROD+KL	-22.30 ± 18.33	$+1.59 \pm 2.87$